A SYSTEM FOR PROCEDURAL GENERATION OF VIRTUAL AGENT ACTIVITY USING LLM-BASED SCENE INTERPRETATION

by

Vinayak Regmi

A Thesis

Submitted to the Faculty of Purdue University

In Partial Fulfillment of the Requirements for the degree of

Master of Science



School of Applied and Creative Computing

West Lafayette, Indiana

December 2025

THE PURDUE UNIVERSITY GRADUATE SCHOOL
STATEMENT OF COMMITTEE APPROVAL

Dr. Christos Mousas, Chair

School of Applied and Creative Computing

Dr. Tim McGraw

School of Applied and Creative Computing

Dr. Stephen Baker

School of Applied and Creative Computing

Approved by:

Dr. Christos Mousas

TABLE OF CONTENTS

LIST OF TABLES 5
LIST OF FIGURES 6
GLOSSARY 7
ABSTRACT 8
1. INTRODUCTION 9
1.1 Large Language Models (LLMs) 9
1.2 Procedural Generation 9
1.3 Agent-Based Systems 10
1.4 Procedural Interaction System 10
1.5 Model Comparison Framework 10
1.6 Problem Description and Research Question 11
1.7 Purpose 11
1.8 1.8 Assumptions 11
1.9 Limitations 12
1.10 Delimitations 12
1.11 Goals 13
2. REVIEW OF LITERATURE 14
2.1 Background 14
2.2 Generative Agents and Narrative Behavior Systems 14
2.3 Multimodal and Visual Storytelling Frameworks 15
2.4 Virtual Character Interaction and Scene Affordance Models 16
2.5 LLM-Driven Interaction in XR and Game Environments 17
2.6 Conclusion 18
3. METHODOLOGY 19
3.1 Methodology Research 19
3.2 System Overview 20
3.3 Pipeline Overview 21
3.3.1 Scene Composition 22
3.3.2 LLM Provide Selection 23

3.3.3 Scene Metadata Generation (Plain Language Description) 25
3.3.4 LLM Query & Procedural Behavior Generation 26
3.3.5 SceneDirector Parsing & Agent Assignment 27
3.3.6 Movement Execution & Animation Playback 27
Layered Animation Handling 27
3.3.7 Finalization 28
3.3.8 Example Output 29
3.3.9 Interaction Types & Handling 29
3.4 Tools & Technologies Used 30
3.5 Estimating the Number of Possible Scenarios 30
3.6 Result Evaluation Plan 31
3.6.1 Testing Scenarios 31
3.6.2 Testing Procedure 32
3.6.3 Data Analysis 32
4. RESULT ANALYSIS 33
4.1 Scenario 1: 1 Object, 1 Agent 33
4.2 Scenario 2: 5 Objects, 1 Agent 34
4.3 Scenario 3: 5 Objects, 2 Agents 36
4.4 Scenario 4: 5 Objects, 5 Agents 38
4.5 Scenario 5: 10 Objects, 5 Agents 40
4.6 Overall Comparison Across Scenarios 42
5. CONCLUSION 45
5.1 Discussion: RQ1 45
5.2 Future Work 46
REFERENCES 47
APPENDIX 50

LIST OF TABLES

Table 1: LLM system prompt. 24

Table 2: Serialized scene description snippet. 25

Table 3: Processing and response time scenario 1. The sample data are measured in milliseconds. 33

Table 4: Processing and response time scenario 2. Data were measured in milliseconds. 35

Table 5: Processing and response time scenario 3. Data are measured in milliseconds. 37

Table 6: Processing and response time scenario 4. Data are measured in milliseconds. 39

Table 7: Processing and response time scenario 5. Data are measured in milliseconds. 41

Table 8: Overall response time over scenarios. 43

LIST OF FIGURES

Figure 1: Overview of the LLM-driven interaction generation pipeline, from user-authored scene setup to the final multi-agent execution in Unity..... 21

Figure 2: Placable scene prefabs selection menu..... 22

Figure 3: An example of a user-assembled scene in Unity..... 23

Figure 4: Placed object edit menu..... 23

Figure 5: Keyframe illustrating the flow of actions..... 29

Figure 6: Boxplots showing response time for scenario 1..... 34

Figure 7: Boxplots showing response time for scenario 2..... 36

Figure 8: Bxplots showing response time for scenario 3..... 38

Figure 9: Boxplots showing response time for scenario 4..... 40

Figure 10: Boxplots showing response time for scenario 5..... 42

Figure 11: Line graph depicting the response time of LLMs over increasing multi-agent scenario complexity..... 44

GLOSSARY

- **Large Language Model (LLM):** A deep learning model trained on large text datasets to understand and generate human-like language.
- **Generative Pretrained Transformer (GPT):** A type of LLM capable of generating structured text outputs based on input prompts.
- **Scene Metadata:** Structured data attached to Unity objects (agents or items) describing their identity, type, and interaction capabilities.
- **SceneDirector String:** A custom-encoded output format used to describe action sequences for agents, generated by LLM.
- **NavMeshAgent:** Unity component that allows agents to navigate a predefined walkable area using pathfinding.
- **Animator Layers:** Unity's animation system feature allows multiple animation tracks to blend (e.g., walking and grabbing simultaneously).
- **Final IK:** A Unity plugin used to control character limbs through inverse kinematics, enabling realistic reach-based interactions.
- **SF Studio Grid Building System:** A unity plugin used to place prefabs in the scene based on a grid system.
- **Prefab:** A reusable Unity object template (e.g., agent or object) with predefined components and metadata.
- **Coroutine:** A Unity method that allows asynchronous timed actions, commonly used for sequential behaviors and animations.
- **Interaction Types:** Predefined categories of actions agents can perform e.g., grab, stationary, normal, or Final IK-based interactions.

ABSTRACT

This thesis presents a system for procedurally generating interactive stories and actions for virtual characters within Unity using large language models (LLMs). Users could drag and drop multiple agents and objects into a scene, with each entity automatically assigned semantic metadata describing its identity, role, and potential interactions. The scene structure is then serialized into a natural language prompt and sent to an LLM-based system, which returns a structured string describing a sequence of actions and interactions among agents and objects. The returned string encodes who performed which actions, when, and how, capturing layered behaviors such as walking, grabbing, sitting, toggling, or performing simple interactions using inverse kinematics (IK) control. A custom parser interprets this string and triggers coordinated agent behaviors using Unity's NavMeshAgent navigation, animation layers, and interaction modules. The system supports simultaneous multi-agent scenes, dynamic object manipulation, and diverse interaction types without the need for manual scripting or pre-authored behavior trees. Designed for ease of use and rapid iteration, the system enables scalable, AI-assisted storytelling suitable for prototyping simulations and cinematic sequences. The performance of the developed system was evaluated using four popular lightweight LLMs, including OpenAI's ChatGPT, Google's Gemini, Anthropic's Claude, and xAI's Grok. Each model's process and response time were measured under multiple complexity scenarios. The collected data were analyzed to compute and compare average response times and consistency across the examined scenarios, highlighting the relative efficiency and suitability of each model for procedural narrative generation. The results demonstrate that large language models can reliably translate high-level scene descriptions into executable multi-agent behaviors suitable for real-time environments. Overall, the system establishes a practical foundation for integrating natural-language narrative planning with procedural character interaction in Unity.

1. INTRODUCTION

The rise of large language models (LLMs) and procedural content generation tools has unlocked new creative opportunities in the domain of real-time interactive storytelling. This thesis presents a novel system for generating procedural stories and actions in the Unity game engine. By combining LLM-based natural language interpretation with a modular agent behavior system, the system allows users to compose scenes by placing agents and objects in a 3D environment, after which the system dynamically generates a coherent narrative and visualizes it through agent animation and interaction.

Unlike traditional behavior tree (BT)-based AI or manually scripted logic [24], [25], [26], this system leverages generative AI to transform high-level prompts into low-level action sequences executed by virtual characters. The resulting architecture supports modularity, extensibility, and ease of authoring, making it suitable for multi-agent simulation and rapid narrative prototyping.

1.1 Large Language Models (LLMs)

LLMs, such as GPT-4.1, are trained on vast text corpora to learn statistical patterns in language and can generate coherent, contextually appropriate text in response to prompts. Their ability to perform in-context learning and plan complex behaviors has made them valuable tools for applications in creative writing, dialogue systems, and planning tasks [19]. When paired with structured interfaces, LLMs can serve as powerful agents for procedural content generation, capable of interpreting and structuring natural language input into structured outputs such as dialogue trees or command sequences [20].

1.2 Procedural Generation

Procedural generation refers to the algorithmic creation of content such as levels, textures, stories, or behaviors without manual authoring. It has been widely used in games to produce large or variable content sets with limited designer input [21]. In narrative domains, procedural systems can generate plots, character arcs, and dialogue, enabling scalable storytelling. This project adopts

a procedural interaction system that generates story beats on the fly based on the semantic configuration of a user-designed scene.

1.3 Agent-Based Systems

In agent-based systems, individual entities are modeled in simulation as autonomous decision-makers with internal logic and goals. These systems are commonly used in crowd simulation [27], [28], training environments, and interactive media to simulate emergent behavior or group dynamics [22]. In our context, each agent is assigned a role and a behavior script derived from the LLM output, with movement and interactions governed by Unity's NavMesh and Animator components. This allows for emergent yet directed interactions based on narrative context.

1.4 Procedural Interaction System

The system introduces a layered interaction model in which user-placed agents and objects are parsed into a semantic scene description. The user selects a LLM model, which then transforms this into behavior strings that encode movement, interaction type (e.g., grab, sit, touch), and timing. These are executed by a SceneDirector module, resulting in a scene that appears authored but is procedurally generated at runtime. This aligns with contemporary efforts in mixed-initiative authoring systems, where artificial intelligence (AI) provides structure and coherence while the user retains creative control [23].

1.5 Model Comparison Framework

To evaluate the performance and practical suitability of various large language models within the proposed system, four transformer-based LLMs were integrated and tested: OpenAI's ChatGPT, Google's Gemini, Anthropic's Claude, and xAI's Grok. Each model was presented with semantically equivalent scene descriptions at controlled levels of complexity, ranging from simple one-agent interactions to multi-agent scenarios with compound object dependencies.

Processing and response time were measured across all models. Data were collected across repeated trials to assess statistical reliability, and the average response was computed to highlight differences in performance. This comparison established a benchmark for lightweight LLMs in

real-time procedural generation contexts, identifying trade-offs between response latency and output quality. The results provided insight into which models are best suited for integration into interactive narrative systems, particularly those requiring rapid, contextually coherent scene generation within Unity.

1.6 Problem Description and Research Question

Traditional narrative systems in Unity rely on predefined timelines, BTs, or hard-coded logic, which are inflexible and time-consuming for the author. They do not scale well to multi-agent scenes or allow for dynamic story generation from arbitrary inputs. Thus, the core research question this work addresses is:

- **RQ1:** How can we bridge natural language-driven narrative planning with real-time behavior orchestration in a modular 3D environment without requiring manual scripting?

1.7 Purpose

The purpose of this project was to design a procedural system that automates the generation of story-driven character actions based on high-level user inputs, such as scene layout design by placing agents and objects. It aimed to provide a generalizable, user-friendly system for narrative experimentation, with potential applications in education, training, and creative media. In addition, this work incorporated a comparative analysis of four transformer-based language models to evaluate their suitability for procedural narrative generation. Their performance was examined in terms of response consistency, processing time, and handling of scene complexity. This dual focus established both a proof-of-concept for interactive generative storytelling and a technical foundation for future research in AI-assisted narrative authoring.

1.8 1.8 Assumptions

The researcher assumes that:

- The agents and objects are properly tagged with semantic metadata.
- The LLM model can produce valid and structured outputs under prompt constraints.

- The scene layout and object placement are handled by the user to ensure spatial plausibility.
- Unity's NavMesh and Mechanim animation systems are configured to correctly execute agent logic.
- The final response time is the sum of the LLMs' internal processing time and response time, recorded as the end-to-end request-to-response time.

1.9 Limitations

While effective within its intended scope, the system has several limitations:

- It relies on external LLM APIs, which introduces latency and requires network access.
- The scene parser depends on consistent formatting of the LLM-received output strings.
- If multiple agents try to access the same object at the same time, it won't be able to parse it or play it correctly.
- When there are many agents and relatively low number of objects the ai tries to access the same object with multiple agents at once.
- Agent reasoning is not autonomous; all logic is derived from the selected LLM without memory or state tracking.

1.10 Delimitations

The following delimitations define the intended scope and boundaries of this project:

- Only five predefined interaction types are supported: normal, grab, stationary, stationary + grab, and basic interactions like reaching to touch the light switch, turning on a washing machine, turning on a television, and turning on a lamp.
- Narratives are limited to short, self-contained sequences involving movement and object interaction.
- All execution is constrained to Unity, using NavMeshAgents for navigation and Animator and FinalIK Plugin for playback of pre-authored animations.
- The researcher assumes structured input formats and object metadata; it does not process freeform or ambiguous language inputs.

1.11 Goals

The core objective of this project is to create a drag-and-drop procedural storytelling system within Unity. Specific goals include:

- Enable users to populate a Unity scene with predefined agents and interactable objects (prefabs).
- Use LLM to generate a structured interaction script based on natural language queries and scene context.
- Parse the script using a Scene Director module that translates the LLM's output into actionable behaviors.
- Allow agents to interact with objects and each other through various types of interactions (e.g., grab, stationary, interact).
- Play out the entire story sequence autonomously once the initial prompt is triggered.
- Use multiple LLMs to run the scenarios and record processing and response times to evaluate and compare further.
- Support extensibility for future logic, such as conditional behaviors, dialog trees, or emotion-based responses.
- Enable fast prototyping of multi-agent simulations without manually scripting each event.

2. REVIEW OF LITERATURE

2.1 Background

Recent advances in artificial intelligence, motion synthesis, and game engines have expanded the possibilities for interactive storytelling and character control. Traditional systems such as behavior trees or manually scripted logic remain reliable for deterministic agent behavior, but they do not scale well to complex, user-defined scenes or dynamically generated narratives. Parallel advances in LLMs have created new opportunities for natural-language-driven planning, while developments in motion learning and object–interaction modeling have improved the realism of character movement.

Prior surveys, such as the work of Kybartas and Bidarra [2], highlight the longstanding divide between symbolic plot-based approaches and reactive agent-level reasoning, suggesting the need for hybrid frameworks that bridge high-level narrative intent with low-level execution. Visual storytelling research, including the work of Liu et al. [1], demonstrates that generative models can create narrative imagery, though not in real time. More recently, research in character motion and human–object interaction, such as the works of Fraga et al. [29], Yu et al. [30], and Bhatt et al. [31], has begun to address how virtual characters can more naturally engage with dynamic environments. Together, these bodies of work contextualize the need for systems that unify LLM-driven planning, real-time interaction, and grounded motion execution inside game engines.

2.2 Generative Agents and Narrative Behavior Systems

Previous work has explored how agents can generate or realize narrative structures. The work of Park et al. [6] simulates human-like behaviors and social dynamics in sandbox environments using memory and planning layers to produce emergent, long-term behavior. While powerful, these systems focus on autonomy and internal state management rather than direct scene execution or animation control.

The work of Ammanabrolu et al. [5] introduces a pipeline to expand symbolic plot graphs into fluent narrative text, a concept aligned with this system's transformation of semantic scene descriptions into structured, LLM-generated action plans. Similarly, the work of Qian et al. [7]

demonstrates how communicative agents can collaborate in software workflows, highlighting the applicability of agent-based modular planning beyond narrative contexts.

The work of Kybartas and Bidarra [2] further categorizes these efforts into plot-based, character-based, and emergent systems, highlighting the importance of narrative coherence and the growing relevance of learning-based approaches. This system fits within this trajectory by using LLMs to semantically interpret a scene and produce interaction logic that is coherent, controllable, and adaptable to modular Unity environments.

Unlike these approaches, Our system does not simulate autonomous reasoning. Instead, it focuses on scene-driven behavior orchestration, generating structured movement and interaction instructions from LLMs and then executing them in Unity.

Moreover, the work of Hu et al. [16] highlights the complexity of scaling BTs for multi-agent narratives. As scene complexity grows, the number of potential transitions and conflict points increases. Our system avoids this by replacing manual tree design with LLM-generated logic strings, encoded as interaction queues with built-in support for grab, stationary, and Final IK-based actions. This provides clarity, modularity, and extensibility while maintaining expressive control.

Behavior authoring in multiactor simulations has also been explored by Kapadia et al. [17], who use a flexible planning framework that enables domain experts to define high-level behaviors while automatically generating actor trajectories within the simulation environment. Their work highlights the tension between specificity and automation in agent behavior systems, a challenge this system similarly addresses by allowing LLM-generated logic to guide low-level execution without requiring detailed manual planning or scripting.

2.3 Multimodal and Visual Storytelling Frameworks

Multimodal systems have explored how natural language prompts can drive animation, visual composition, or asset generation. The work of Li et al. [10] demonstrates how hierarchical LLM agents can convert short prompts into complex, multi-modal digital stories. However, its output is limited to rendered assets rather than real-time interaction. The work of He et al. [14] applies text-to-animation synthesis, enabling prompt-driven animation clips for pre-rigged characters, while the work of Liew et al. [4] performs high-fidelity video edits using temporally

coherent latent diffusion. The work of Kapadia et al. [3] provides a GUI-based system for visually constructing story sequences through keyframes and narrative arcs.

While these frameworks generate impressive outputs, they lack runtime integration with character controllers, environmental logic, or dynamic object interactions. In contrast, this system integrates directly into the Unity runtime. The LLM's output is not just a narrative plan but a fully executable instruction set parsed by the SceneDirector module and mapped onto game objects with unique IDs and semantic tags, enabling interactive in-engine storytelling without authored animation timelines.

2.4 Virtual Character Interaction and Scene Affordance Models

A growing body of research focuses on how virtual characters perceive, interact with, and move within complex 3D environments. These works complement narrative-generation and multimodal systems by addressing how agents can perform physically grounded interactions, navigate object-rich scenes, and realistically synthesize motion trajectories.

The work of Fraga et al. [29] models interaction dynamics by learning motion graphs combined with neural fields, enabling agents to perform context-appropriate motions that adapt to spatial constraints. Their approach demonstrates how learned interaction manifolds can replace hand-authored animation transitions, offering flexibility in environments with diverse object configurations. Although their system focuses on motion quality and physical plausibility rather than narrative planning, it highlights the importance of data-driven interaction modeling for believable agent behavior.

Similarly, the work of Yu et al. [30] examines object interaction landscapes and proposes methods for understanding the relationship between human movement and surrounding affordances. By analyzing interaction fields around objects, their method enables virtual characters to predict feasible and semantically aligned interaction poses. This contributes to a deeper understanding of how agents should position themselves relative to objects, a challenge also presented in narrative-driven systems that must translate high-level intent into spatially coherent actions.

The work of Bhatt et al. [31] introduces a generative motion-planning framework for human–object interaction that combines learned representations with sampling-based planning. Their method produces goal-directed interactions that account for both motion feasibility and

environmental context. While oriented toward physical manipulation tasks, the underlying idea of using generative models to inform multi-step interaction sequences parallels the need in narrative systems to convert textual descriptions into structured, executable behavior plans.

Together, these works illustrate an expanding research area that seeks to ground virtual character behavior in realistic spatial and physical constraints. While they do not address narrative sequencing or multi-agent story logic, they highlight techniques and principles that can inform the development of future storytelling systems where LLM-generated plans must integrate closely with physically plausible motion and object interactions.

2.5 LLM-Driven Interaction in XR and Game Environments

LLMs have also been incorporated into immersive environments to enhance interactivity. The work of Normoyle et al. [9] uses GPT-3.5 to control emotional body language in NPCs, offering expressiveness beyond pre-scripted dialogue. This system supports similar forms of physical interaction logic and coordination, enabling behaviors such as walk-and-grab or sit-and-type that blend motion with environmental feedback.

The work of Li et al. [13] and Li et al. [11] explored adaptive storytelling based on scene semantics or geographic data, a form of environmental storytelling where narrative progression is tied to spatial conditions. While innovative, these frameworks are context-reactive and do not offer generalized runtime orchestration of characters or scenes.

The work of Rychert et al. [15] integrates GPT into a VR escape-room experience, but GPT functions primarily as a guide rather than a controller of in-scene logic or agent behavior. Likewise, the work of Ning and Pei [8] addresses spatial rearrangement for safety rather than narrative planning or execution.

In contrast, this system uses LLMs as a story logic engine, parsing scene structure into action sequences and translating them into coroutine-based Unity behaviors, supporting normal, grab, stationary, and basic IK interactions. Complementary to this, the work of Chang et al. [18] introduces a semantic injection mechanism that improves text-to-motion alignment. While their system focuses on motion fidelity and token alignment, the present system emphasizes interaction logic and real-time execution within a 3D environment.

2.6 Conclusion

Prior research relevant to procedural storytelling and virtual agent behavior can be grouped into four main areas. The first includes narrative logic and generative agent systems that address symbolic planning, autonomy, and long-term emergent behavior [2], [5], [6], [7], [16], [17]. The second encompasses multimodal animation and visual storytelling frameworks that translate prompts into visual sequences using techniques such as keyframe generation, diffusion models, or pre-authored animation synthesis [1], [3], [4], [10], [14], [18]. The third area focuses on LLM-driven interaction and spatial storytelling in XR and game environments, where language models are used to interpret scenes, generate context-aware actions, or augment user-driven narratives [8], [9], [11], [13], [15].

A fourth and increasingly significant research direction involves motion- and interaction-driven methods for virtual character control. This includes approaches that learn interaction dynamics using neural motion fields [29], model object-affordance relationships to guide character poses [30], or employ generative motion-planning techniques for human–object interaction [31]. While these works primarily emphasize physical plausibility and motion quality, they highlight techniques that may inform future systems seeking deeper integration between narrative planning and grounded character behavior.

Building on advances across these four areas, this thesis presents an integrated system for procedural multi-agent action generation in Unity. The system enables users to arrange prefabricated agents and objects, extract semantic scene metadata, and generate structured behavior plans through a large language model. These plans are encoded using a custom SceneDirector syntax and executed in real time through coroutine-based interaction logic, NavMeshAgent navigation, and layered animation blending supporting concurrent interactions such as stationary and grab behaviors. By replacing manually authored behavior trees or state machines with scene-aware LLM-generated logic, the system supports rapid prototyping, reduces authoring overhead, and enables non-technical users to design coherent, interactive multi-agent stories without extensive scripting or animation planning.

3. METHODOLOGY

3.1 Methodology Research

Procedural story generation in interactive environments builds on established methods in game AI, character animation, and narrative planning. Traditional behavior models, such as BTs and state machines, offer deterministic control but become increasingly complex in narrative-rich or emergent scenarios that demand scalable agent-object coordination.

Recent work has explored symbolic planning, plot graph expansion, and memory-based generative agents [5], [6], [7], emphasizing internal reasoning and social dynamics. Other approaches focus on text-to-animation or multimodal storytelling via LLMs [1], [10] but often lack integration with real-time runtime execution environments such as Unity.

This thesis introduces a system that embeds LLM-generated logic directly into Unity's runtime, combining LLM-driven natural language parsing with structured prefab metadata. In contrast to static synthesis systems such as Intelligent Grimm [1], this system produces interactive story sequences executed through Unity's animation and navigation systems. Rather than relying on reinforcement learning, symbolic inference, or hard-coded trees, the system leverages structured prompt design and scene-aware data serialization to generate interaction plans as logic strings. These are parsed into modular, coroutine-driven behaviors by a custom SceneDirector module.

To evaluate the system, a comparative model-based analysis was conducted using four transformer-based large language models. Each model was integrated into the procedural generation pipeline and tested at controlled levels of scene complexity. Equivalent semantic scene descriptions were sent to all four models. Their responses were analyzed across multiple performance dimensions: processing and response time, [1] and instructional adherence and coherence of the generated sequence. This analysis provided insight into the relative efficiency and reliability of each model when used for real-time procedural narrative generation in Unity, establishing a practical benchmark for future AI-assisted storytelling systems.

[1] Since the processing internally done by the LLM isn't accessible to the API directly, we are timing the combined end to end response time, from sending the request to receiving the response.

3.2 System Overview

This thesis presents a procedural generation system for story and action sequences within Unity, using LLM-based logic generation and metadata-driven prefab composition. The system does not require manual scripting, BT construction, or domain-specific modeling tools. It enables users to create complex, emergent story scenarios simply by placing prefabricated agents and objects into a scene and triggering the story generation pipeline. The system comprises the following integrated components:

- **Scene Composition**: Users drag human agents and interactable objects (e.g., mugs, computers, chairs) into a Unity environment using the Grid Building Plugin. Each object includes attached metadata via custom scriptable objects and MonoBehaviours.
- **LLM Provider Selection**: Users select an LLM provider to generate the interactions between ChatGPT, Gemini, Claude, and Grok.
- **Scene Metadata Serialization**: A SceneSerializer component extracts metadata from all agents and objects in the scene and formats it as a structured plain-language scene description. This includes agent IDs, object types, object interaction types, semantic tags, and the world-space positions of objects and agents (*see Table 2*).
- **LLM Prompting**: The structured scene description is sent to the selected LLM via the API along with a predefined prompt. The prompt is designed to elicit structured responses in the SceneDirector format, a syntax that encodes the sequence of actions, interaction types, durations, and motion speeds for each agent.
- **SceneDirector Parsing**: The SceneDirector module parses the LLM's output, maps it to in-scene object references, and assigns ordered Destination queues to each agent. Each Destination specifies the interaction type (i.e., normal, grab, stationary, or basic), the animation duration, and movement speed.
- **Interaction Types:** The system supports four categories of agent–object interactions: *normal interactions* performed at a destination, *grab interactions* involving object pickup and transport, *stationary interactions* that hold the agent in place for context-specific animations, and basic *IK-driven interactions* that trigger simple object responses such as toggling lights or activating devices.
- **Agent Execution**: Each agent executes its interaction queue using Unity's NavMeshAgent for pathfinding and Animator for animation playback. Layered

animation handling allows simultaneous execution of grab (Layer 1) and stationary (Layer 0) animations, coordinated through coroutines.

- **Finalization**: After all destinations are executed, the agent returns to an idle state and stops pathfinding. Animators are reset to a base locomotion blend tree.

The system presents an end-to-end mechanism that converts scene composition into executable multi-agent behaviors through structured LLM-generated instructions. Its modular design supports scalable interaction logic and flexible extension of narrative capabilities.

3.3 Pipeline Overview

The system generates procedural multi-agent sequences by allowing users to assemble scenes from prefabricated agents and objects within the Unity environment. The overall pipeline proceeds from scene composition to the execution of dynamic, LLM-driven animation behaviors.

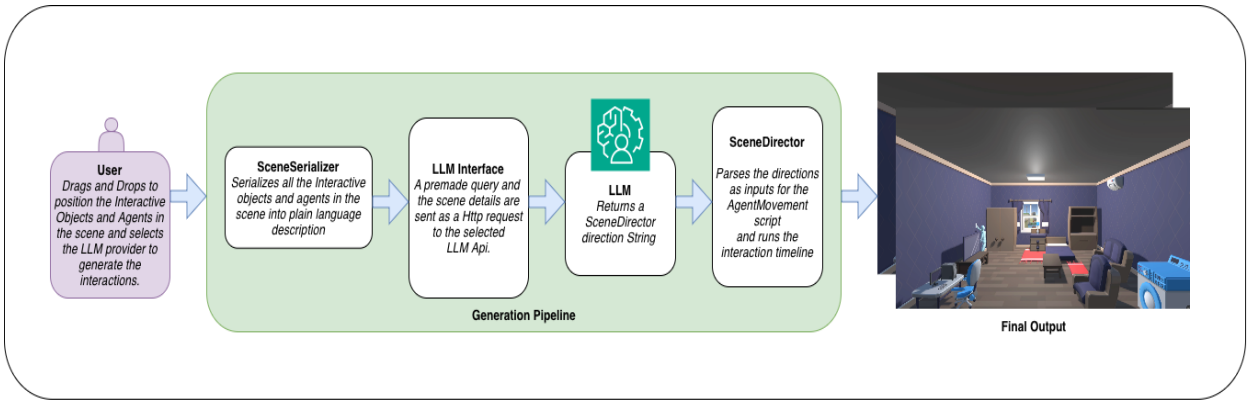


Figure 1: Overview of the LLM-driven interaction generation pipeline, from user-authored scene setup to the final multi-agent execution in Unity

Figure 1 illustrates the end-to-end pipeline in which user-placed agents and objects are serialized into a plain-language scene description, sent through an LLM interface, and converted into a structured SceneDirector instruction string. This output is then parsed and executed within Unity, producing real-time multi-agent interactions based on the generated action plan.

3.3.1 Scene Composition

Scene setup begins with the placement of prefabricated elements drawn from the object directory. These include human agents equipped with navigation, animation, and behavior components; interactable objects that respond to animations or context-specific actions, such as sitting, grabbing, or activating devices; and non-interactable objects that serve purely as environmental or decorative elements within the scene. These objects are placed into the grid system by selecting them through a selection menu, as seen in Figure 2.



Figure 2: Placable scene prefabs selection menu

Each placed prefab is associated with a corresponding metadata wrapper that encodes semantic and functional information. Human agents include a SelfExplainer component specifying attributes such as name, identifier, position, and semantic tags (e.g., “child,” “worker”). Interactable objects are linked to an InteractableObjectExplainer ScriptableObject that records the object’s identifier, animation set, supported interaction types (i.e., normal, grab, stationary, and basic), semantic descriptors, and spatial position. This metadata forms the basis for later serialization and LLM-based behavior generation. An example of an assembled scene is shown in *Figure 3*. Finally, once an object is placed in the scene, it can be repositioned, rotated, or removed through the object editing interface (*see Figure 4*).



Figure 3: An example of a user-assembled scene in Unity.



Figure 4: Placed object edit menu.

3.3.2 LLM Provide Selection

The user can select a LLM provider between ChatGPT (*gpt-4.1-mini*), Claude (*claude-sonnet-4-5*), Gemini (*gemini-2.5-flash*), and Grok (*grok-4-1-fast).* The selected provider is used when creating the interactions. The scene metadata (*see Section 3.3.3*) is sent to the selected LLM, which returns the interaction string *(see Section 3.3.4*) that can be parsed by the SceneDirector. All the LLMs are conditioned with the same system prompt as seen in *Table 1* to tailor their responses to our application's needs.

Table 1: LLM system prompt.

```
 1. You are a procedural story generation assistant. Your task is to convert a formatted scene description into a Unity-compatible SceneDirector instruction string used by a game system.
 2.
 3. You will:
 4. - Create a short, coherent story using the provided human agents and interactable objects.
 5. - Each agent (e.g., H_1, H_2, H_3...) must act independently, and **no two agents may use the same object at the same time**.
 6. - Grabbed objects are destroyed after use and cannot be reused.
 7. - **Grab** and **Stationary** interactions are compatible and may be layered in sequence (e.g., carrying an object and sitting down).
 8. - For all other interactions, objects must be freed before another agent may use them.
 9. - **Overestimate occupancy**: Account for time to walk to the object + interaction duration when tracking object availability.
10. -Speed can range from 1.0-4.0
11. -duration can range for 2 to 16 seconds so use the length that fits your story telling
12. - and for basic interaction keep the max limit at 5 and min at 3
13.
14. Each human agent must be represented as:
15. HUMAN_ID{ObjectID_1(T/F, DURATION, SPEED, GRAB_TF, STATIONARY_TF, BASIC_TF), ObjectID_2(...)}
16.
17. Definitions:
18. - `interact (T/F)`: Always **T** if any of grab, stationary.
19. - `duration`: interaction duration (e.g., 3.5 seconds).
20. - `speed`: Movement speed (e.g., 1.0).
21. - `grab`: True if the agent picks up and carries the object.
22. - `stationary`: True if the agent performs a stationary action like typing, sitting, or sleeping.
23. - `basic`: True for short Final IK-style interactions (e.g., flipping a switch, turning on a light or machine).
24.
25. □ Output **only** the SceneDirector string and nothing else.
26.
27. ### Example Output:
28. If H_1 turns on a light switch (Obj_1), then sits at a desk (Obj_2), your response should be:
29. H_1{Obj_1(T, 2, 1, F, F, T), Obj_2(T, 5, 1, F, T, F)}
30.
31. You will be sent:
32. - A plain-language scene description.
33. - A list of available agents and objects (with IDs and features).
34. - Your response must only contain the properly formatted SceneDirector string.
35.
36. Note: There may be duplicate objects e.g. multiple beds, that is so that multiple agents can interact with different ones, make it make sense story wise, one agent may be able to interact with
```

```
multiple light switches, but it doesn't make sense if an agent sleeps on one bed and gets up and sleeps on the other so be reasonable with the story and pay attention to these scenarios. Also you have the option to maybe move to some object and not interact with it as the output format suggests, dont always opt for it but in multi agent scenario, if all objects are occupied you may opt for this
37. - Also remember to place a comma after each human's complete entry before starting the next human's entry
38. Eg: H_1{...}, H_2{...}
39. - Also remember to never overlap actions between two different agents, eg going to sit on a couch while other is still sitting, going to interact with a light switch while the other is also doing the same
40. Scene Note: All Lights are currently off, take that into consideration
41. you can turn it on, do stuff and later turn it off, just a hint
42.
43. Final Tip: Your main objective is to create a story and play it out based on what you have, so if some objects are not needed in your story you don't have to always use it, just make it look like a relevant story
```

3.3.3 Scene Metadata Generation (Plain Language Description)

Once the scene is populated, a custom Scene Serializer script is triggered. This system scans the scene to collect metadata from human agents and interactable objects, then converts it into a structured string. This plain language ensures the LLM can interpret the scene semantically before generating structured behavior. See *Table 2* for an example of a serialized scene description.

Table 2: Serialized scene description snippet.

```
1. Scene Description:
 2. ----------
 3. Actors:
 4. ----------
 5. Name: Guy
 6. ID: H_1
 7. Tags: male, college student, casual, claustrophobic
 8. Position: (-0.36, 0.11, -6.12)
 9. ----------
10. ----------
11. Interactable Objects:
12. ----------
13. Object ID: Obj_5
14. Name: Chair
15. Is Grabbable: No
```

```
Is Stationary: Yes
Is Stationary Compatible: No
Is Basic Interaction :No
Tags: chair, sit, stay, relax
Position: (-1.18, 0.23, -5.55)
----------
Object ID: Obj_1
Name: Computer
Is Grabbable: No
Is Stationary: Yes
Is Stationary Compatible: No
Is Basic Interaction :No
Tags: work, play games, desktop, office work
Position: (0.70, 0.26, -5.46)
----------
----------
     END
----------

```

3.3.4 LLM Query & Procedural Behavior Generation

The plain-language scene description is combined with a predefined internal prompt (i.e., crafted to elicit structured movement instructions) sent via API to the selected LLM, which returns an encoded action plan for each agent. The returned response is formatted using a custom syntax known as the SceneDirector string, which follows the pattern:

$$A_1\{O_1(I, D, S, G, St, B)\}, O_2(\ldots)\}, A_2\{\ldots\}$$

Where each entry includes:

- A: Agent ID
- O: Object ID
- I: Interact (T/F)
- D: Interaction Duration
- S: Movement Speed
- G: Grab (T/F)
- St: Stationary (T/F)
- B: Basic Interaction(T/F)

Example: *H_1{Obj_1(T,2,1.5,F,T,F), Obj_2(F,1,1.5,F,F,T)}, H_2{…*

In this scenario, Agent H_1 interacts with object Obj_1 for 2 seconds by moving to it at 1.5x speed. The interaction is a stationary interaction, then Agent H_1 moves to Obj_2 at 1.5x speed. Although it is a basic interaction, the agent doesn't interact with it and stays at its location for 1 second. While this is going on, Agent H_2 is performing its interactions in parallel.

3.3.5 SceneDirector Parsing & Agent Assignment

The SceneDirector component parses the LLM's response, identifies each human agent by ID, and assigns each agent a queue of Destination structs, defining the movement and interaction instructions in order. Each Destination contains a reference to the interactable object, flags for interaction, grab, basic, and stationary types, and speed and animation duration for each interaction.

3.3.6 Movement Execution & Animation Playback

The agent's Movement script executes the destination queue using Unity's NavMeshAgent for pathfinding and Animator for animation control. Each interaction type is handled using dedicated coroutine logic:

- **Normal Interaction:** Move to object → Stop → Play animation → Wait → Move on.
- **Grab Interaction:** Move to object → Grab and attach object → Play grab animation while moving to next → Drop when done.
- **Stationary Interaction:** Move to point → Stop → Play long idle animation (e.g., sleeping, sitting) → Wait → Continue.
- **Basic Interaction:** Triggers IK-based interaction using Final IK's InteractionSystem**.** Often linked with other scripts (e.g., toggling a light, turning a washing machine on).

Layered Animation Handling

When grab and stationary interactions occur concurrently, the animation system resolves them through layered blending. Grab animations are assigned to the upper-body layer (Layer 1), while stationary actions are maintained on the lower-body layer (Layer 0), allowing both to play simultaneously without conflict. Timing is coordinated so that the carried object remains attached

until the grab sequence fully completes, ensuring consistent visual behavior even when animation durations overlap.

3.3.7 Finalization

Once all interactions for a character are completed, the character returns to an idle or default state, the NavMeshAgent is stopped, and animations are reset to "Locomotion."

See Section 3.3.8 for an example scenario. While this is happening in one agent, if there are other agents in the scene, the same processes are happening for all of them.

3.3.8 Example Output

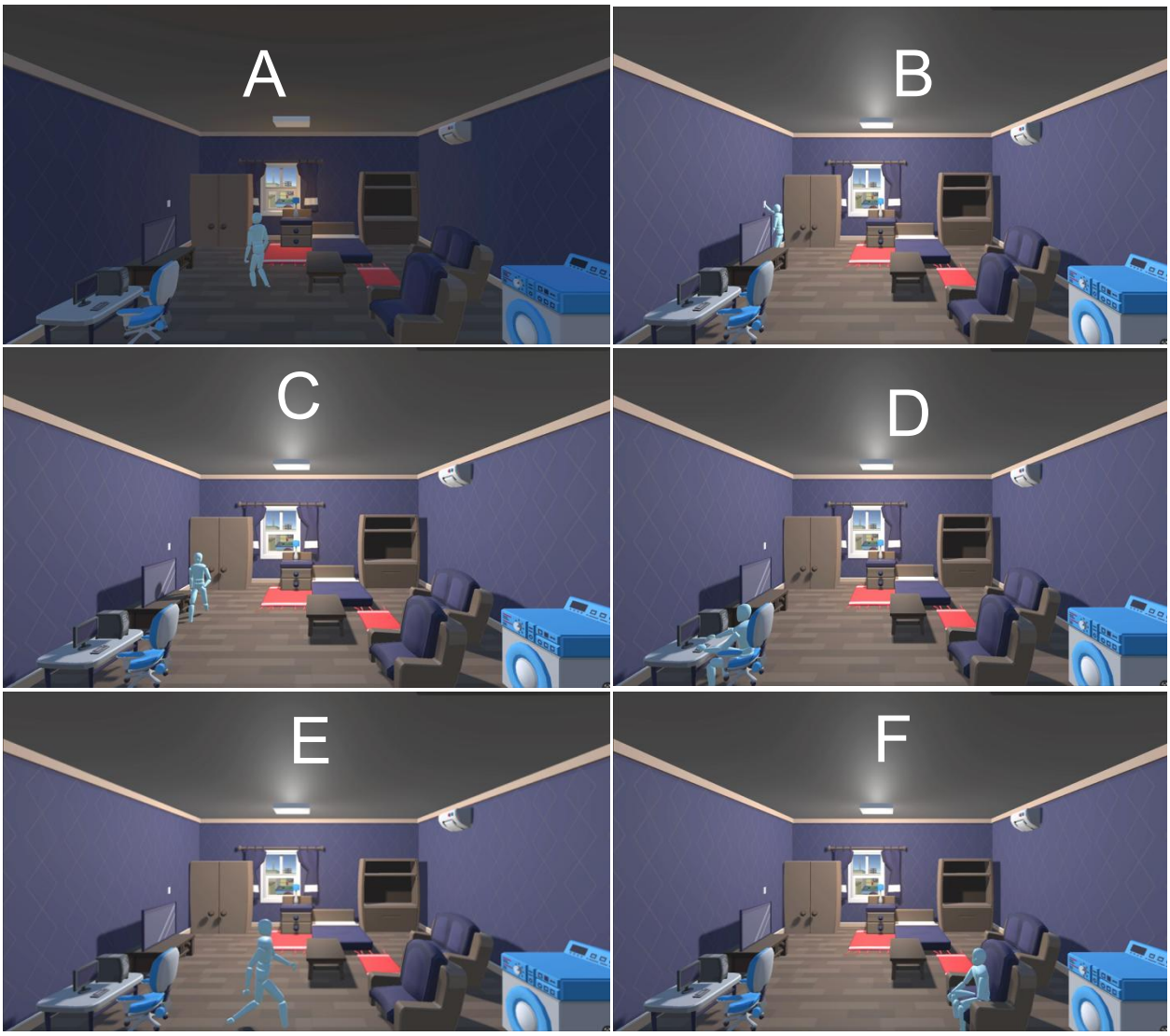


Figure 5: Keyframe illustrating the flow of actions.

The example in Figure 5 illustrates the flow of the final output. The agent moves to the light switch (A), the agent toggles the light switch (B), the agent moves to the next destination (C), the agent sits down and uses the computer (D), the agent moves to the next destination (E), and the agent sits down on the chair (F).

3.3.9 Interaction Types & Handling

The system supports four primary interaction categories: normal, grab, stationary, and basic; each corresponding to a distinct mode of agent behavior (see *Section 3.3.6* for execution details). Normal interactions involve navigating to an object and performing a full-body animation, while grab interactions additionally attach the object so it can be carried to subsequent destinations.

Stationary interactions hold the agent at a location for a sustained contextual animation, and basic interactions trigger short IK-driven contact events that activate object-specific behaviors. All interaction types rely on coroutine-based timing and animation control, allowing them to integrate smoothly with navigation, layered animation blending, and multi-agent sequencing without reintroducing the operational detail described earlier.

3.4 Tools & Technologies Used

The system was developed in Unity 2022.3.9, which served as the primary environment for scene construction, runtime execution, and agent–object interaction management. All system logic was implemented in C# using JetBrains Rider as the main development environment. Four lightweight LLMs, ChatGPT (GPT-4.1 mini) from OpenAI, Claude (Claude Sonnet 4.5) from Anthropic, Gemini (Gemini 2.5 Flash) from Google, and Grok (Grok 4.1 Fast) from xAI, were integrated through their respective APIs to convert serialized scene descriptions into structured interaction logic. The animation system incorporated Final IK for inverse-kinematics-based adjustments, enabling precise pose alignment during basic interactions. Scene construction was supported by the SF Studio Grid Building System, allowing grid-aligned placement of agents and objects. Animation assets were sourced from Mixamo, supplying humanoid clips for idle, interaction, and grab behaviors. ScriptableObjects were used to store metadata for each prefab, ensuring scalable, semantically consistent configuration of agents and interactable objects throughout the scene.

3.5 Estimating the Number of Possible Scenarios

To quantify the diversity of possible story outcomes in this procedural generation system, we analyzed the factors contributing to scenario variation. Unlike traditional branching logic systems, our approach allows for dynamic combinations of agents, objects, and interaction contexts. While each object in the scene supports only one type of interaction (e.g., grab, stationary, or basic), the richness of the system arises from how these interactions are sequenced, layered, and distributed across agents. Interactions can vary in duration, occur at different locations, and may overlap when compatible (e.g., grab and stationary interactions).

Additionally, the LLM-based story planner considers the contextual positioning of each object and agent to compose semantically coherent sequences. Considering these variables, we derive the following formula to estimate the number of potential interaction scenarios:

$$Total\ Scenarios = (m \cdot v \cdot d)^n$$

Where:

- **m**: number of interactable objects,
- **v**: number of spatial placements or contextual variants per object,
- **d**: number of distinct duration or timing variations for interactions,
- **n**: number of human agents in the scene.

This formulation captures not just object-agent assignments, but also spatial arrangement, interaction timing, and action sequencing, all of which contribute to generating emergent narratives using LLM-driven story planning.

3.6 Result Evaluation Plan

The evaluation focuses on benchmarking the performance of four LLMs, *ChatGPT (gpt-4.1-mini), Claude (claude-sonnet-4-5), Gemini (gemini-2.5-flash), and Grok (grok-4-1-fast)* when integrated into the procedural story-generation system. Each model was tested across a fixed set of scene-complexity scenarios to measure response latency, consistency, and validity of output.

3.6.1 Testing Scenarios

Five controlled test scenarios were designed to represent increasing narrative and computational complexity:

1. 1 Object, 1 Agent
2. 5 Objects, 1 Agent
3. 5 Objects, 2 Agents
4. 5 Objects, 5 Agents
5. 10 Objects, 5 Agents

Each configuration was converted into a structured natural-language prompt describing spatial relationships, object affordances, and agent roles. All models received the same prompt text to ensure evaluation consistency.

3.6.2 Testing Procedure

For each testing scenario, an identical natural-language prompt was submitted to all four language models in sequence via the automated testing interface. The system measured the response time from the initiation of the request, LLM processing, to the receipt of a complete output, and recorded it in milliseconds. To reduce the influence of transient network variability, each model was tested five times per scenario, and the average value was used in the final analysis.

In addition to timing measurements, each response was examined for structural validity, ensuring that the generated output adhered to the expected formatting required by the Unity parser. Responses that met these criteria were further reviewed for adherence to basic instruction and for the coherence of the resulting action sequence, allowing the evaluation to assess whether increasing scene complexity affected the models' ability to produce logically consistent and syntactically usable behavior plans. Only responses satisfying these validity requirements were retained in the final dataset.

3.6.3 Data Analysis

Following data collection, the recorded response times were organized into a comparative dataset structured by model and scenario. For each model-scenario pair, the five timing samples were used to compute descriptive statistics, including the mean response time (*processing* + *response*) and the trial-to-trial variance, enabling a quantitative assessment of model efficiency and stability. Scenario-level timing patterns were examined through histograms to illustrate distributional characteristics, while aggregated line plots were generated to compare overall scaling behavior as scene complexity increased. In parallel with the timing analysis, the retained outputs were reviewed qualitatively to confirm that they were structurally valid, consistent with the prompt constraints, and fully parsable by the SceneDirector. These statistical and observational evaluations served as the basis for the performance comparison presented in Chapter 4.

4. RESULT ANALYSIS

The evaluation produced quantitative timing measurements and qualitative assessments of output validity across all scenarios. This chapter presents the observed outcomes of the model comparisons, with each scenario summarized in terms of processing time, variability, and the correctness of the generated SceneDirector strings.

4.1 Scenario 1: 1 Object, 1 Agent

In the simplest scenario, involving a single agent and a single interactable object, all four models produced valid, coherent SceneDirector strings that adhered to the prompt constraints and were successfully parsed by the Unity system. Representative outputs for each model are provided in Appendix A1.1.

Table 3: Processing and response time scenario 1. The sample data are measured in milliseconds.

	ChatGPT	**Claude**	**Gemini**	**Grok**
	911.41	2877.62	3388.15	3419.33
	600.59	3658.09	2487.83	4994.83
	706.52	3118.99	2192.08	5019.12
	904.17	2847.41	3929.93	4853.99
	810.76	3822.99	2702.12	3625.80
Average (s)	0.79	3.27	2.94	4.38
SD (s)	0.13	0.45	0.71	0.79

As observed from *Table 3,* in terms of timing, ChatGPT (gpt-4.1-mini) was the fastest model in this scenario, with a mean response time of 0.79 s (SD = 0.13 s). Gemini (gemini-2.5-flash) and Claude (claude-sonnet-4-5) followed with mean times of 2.94 s (SD = 0.71 s) and 3.27 s (SD = 0.45 s), respectively. Grok (grok-4-1-fast) was the slowest, with a mean response time of 4.38 s (SD = 0.79 s). The histogram for this scenario (*see Figure 6)* shows relatively tight clustering for ChatGPT and Claude, with slightly broader distributions for Gemini and Grok. Overall, all four models exhibited stable behavior at this lowest complexity level.

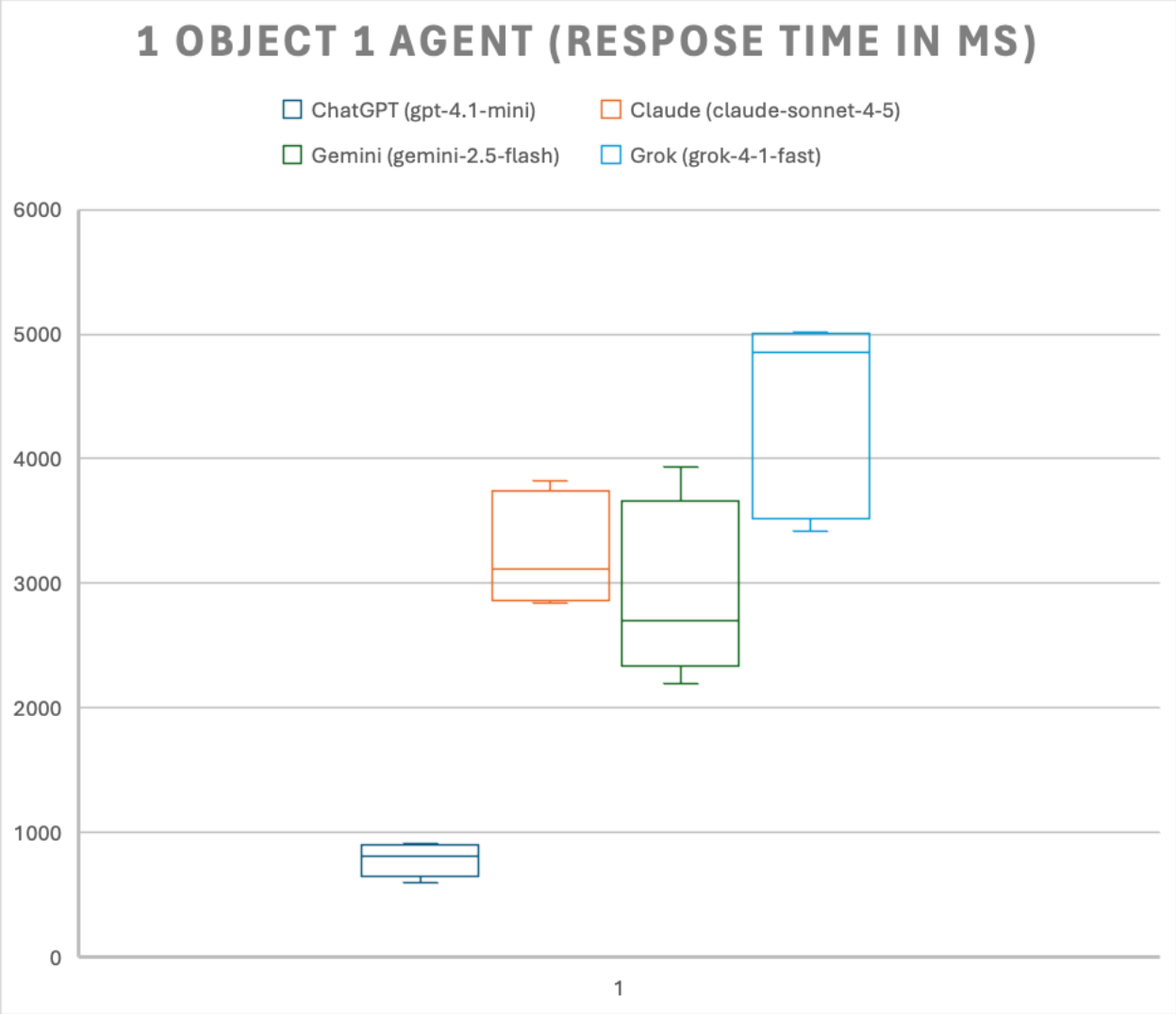


Figure 6: Boxplots showing response time for scenario 1.

4.2 Scenario 2: 5 Objects, 1 Agent

In the second scenario, a single agent interacted with a more populated environment containing five objects. All models again produced structurally valid and semantically coherent responses consistent with the scene description and system prompt, and all outputs were fully parsable by the SceneDirector. Representative outputs for each model are provided in Appendix A 1.2.

Table 4: Processing and response time scenario 2. Data were measured in milliseconds.

	ChatGPT	**Claude**	**Gemini**	**Grok**
	1702.42	4758.32	8229.56	34853.24
	1476.03	3723.69	5528.3	33019.62
	1160.92	4035.54	8795.91	24000.15
	1687.81	4134.52	4229.67	25644.57
	1570.87	5778.27	7723.40	24410.63
Average (s)	1.52	4.49	6.99	28.38
SD (s)	0.22	0.81	1.94	5.14

As shown in Table 4, response times increased relative to Scenario 1 across all four models. ChatGPT remained the fastest, with a mean response time of 1.52 s (SD = 0.22 s). Claude's mean latency rose to 4.49 s (SD = 0.81 s), while Gemini's average response time increased to 6.90 s (SD = 1.94 s). Grok showed a substantial increase in latency in this scenario, with a mean of 28.39 s (SD = 5.14 s). The timing histogram for this scenario (*see Figure 7*) highlights this divergence: ChatGPT and Claude maintained relatively compact distributions, Gemini showed greater spread, and Grok exhibited the highest mean latency and a noticeably larger variance.

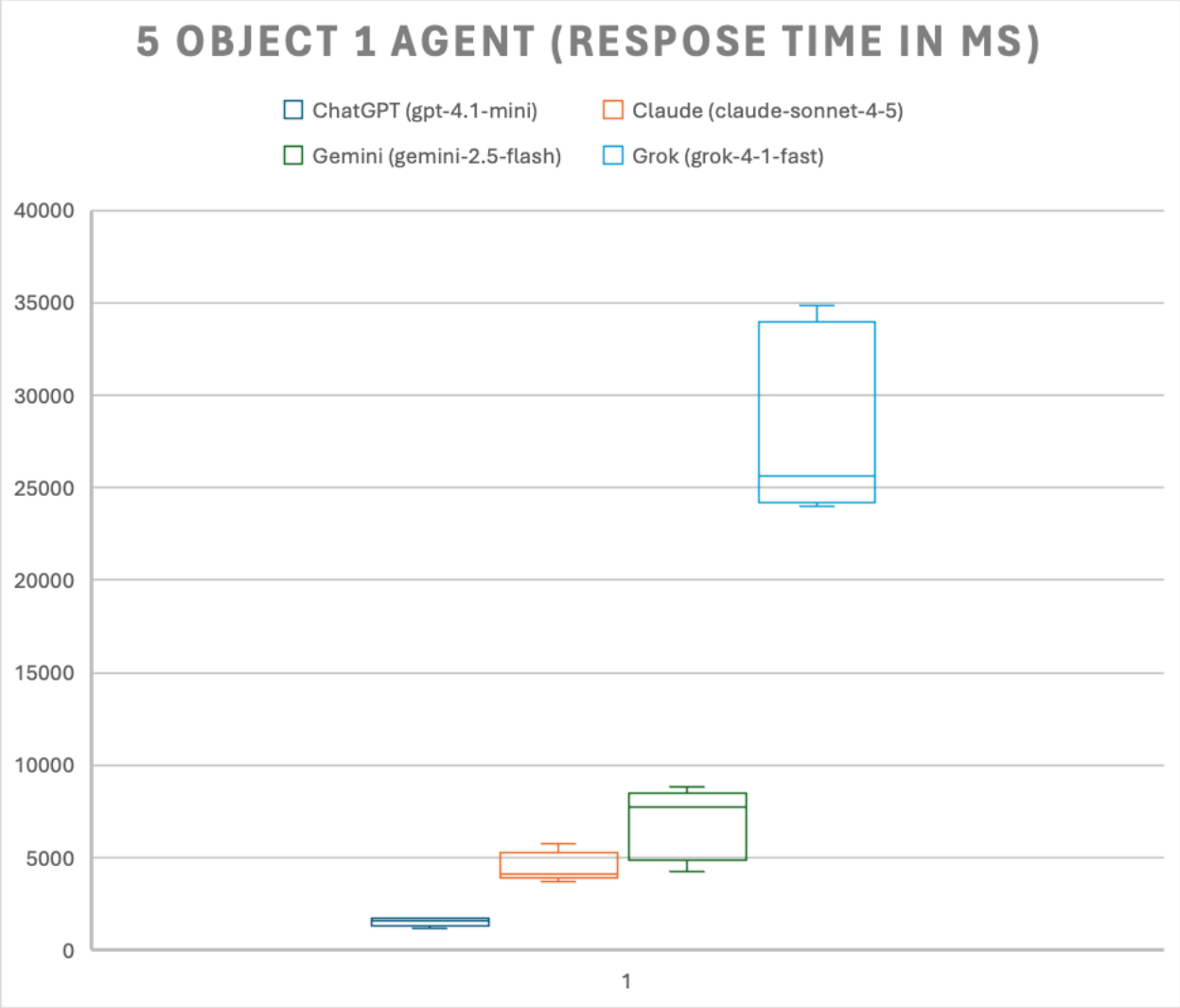


Figure 7: Boxplots showing response time for scenario 2.

4.3 Scenario 3: 5 Objects, 2 Agents

The third scenario introduced multi-agent coordination, with two agents and five objects. As in the previous cases, all four models generated structurally valid outputs, adhered to the formatting requirements of the Unity parser, and produced coherent multi-agent interaction sequences aligned with the textual scene description. No responses were discarded for structural or semantic reasons. Representative outputs for each model are provided in Appendix A 1.3.

Table 5: Processing and response time scenario 3. Data are measured in milliseconds.

	ChatGPT	**Claude**	**Gemini**	**Grok**
	5667.50	4602.77	9177.25	21492.38
	2497.10	4284.27	8006.25	16733.26
	3524.22	4098.37	16785.14	15735.47
	2116.56	4478.24	4821.02	21623.29
	3698.07	5672.97	6005.51	27235.32
Average (s)	3.5	4.63	8.96	20.56
SD (s)	1.38	0.62	4.69	4.60

As observed in *Table 5,* under this increased complexity, ChatGPT's mean response time rose to 3.50 s (SD = 1.38 s), representing its slowest average across the tested scenarios and reflecting the added burden of planning for an additional agent. Claude's mean latency was comparatively stable at around 4.63 s (SD = 0.62 s), showing only a modest increase relative to the previous scenario. Gemini's average response time increased to 8.96 s (SD = 4.69 s). In comparison, Grok's mean latency remained high at 20.56 s (SD = 4.60 s). The histogram for this scenario (*see Figure 8*) suggests that Claude remained relatively consistent. In contrast, ChatGPT, Gemini, and Grok each exhibited broader distributions, indicating greater variability in response time when multi-agent reasoning was required.

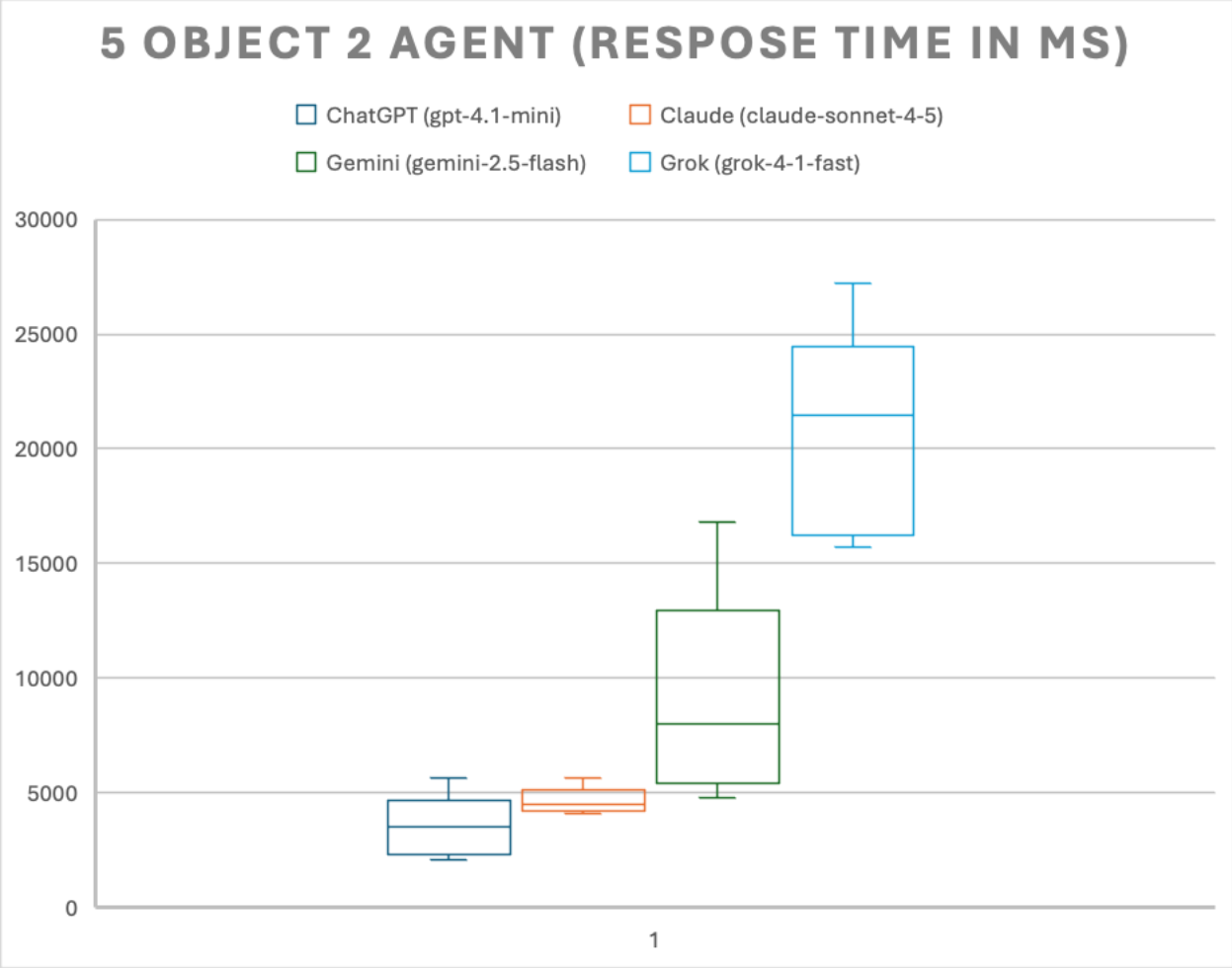


Figure 8: Bxplots showing response time for scenario 3.

4.4 Scenario 4: 5 Objects, 5 Agents

The fourth scenario further increased complexity by involving five agents and five objects, requiring the models to coordinate a denser multi-agent plan. All four models still produced responses that were structurally correct, coherent with the prompt, and successfully parsed by the SceneDirector into executable interaction sequences. Representative outputs for each model are provided in Appendix A*1.4*.

As mentioned previously in the limitations, upon further tests, there was one scenario where the agents attempted to use the same object at the same time, this can be attributed to high number of agents and relatively low number of objects.

Table 6: Processing and response time scenario 4. Data are measured in milliseconds.

	ChatGPT	Claude	Gemini	Grok
	3099.84	5365.63	15095.61	140986.42
	2599.20	4753.97	17339.51	21986.52
	2344.15	5473.54	17758.68	44176.22
	2413.48	5972.75	17756.54	34911.91
	2172.31	5258.69	10897.45	49054.03
Average (s)	2.53	5.36	15.77	58.22
SD (s)	0.36	0.44	2.94	47.40

As shown in *Table 6,* in this configuration, ChatGPT achieved a mean response time of 2.53 s (SD = 0.36 s), faster on average than in the two-agent case despite the higher number of agents and objects, likely reflecting normal variability at this scale rather than strictly monotonic growth. Claude's mean response time increased to 5.36 s (SD = 0.44 s). Gemini showed a more pronounced latency increase, with a mean of 15.77 s (SD = 2.94 s). Grok exhibited the highest latency and the largest variability, with a mean response time of 58.22 s and a very large standard deviation (SD = 47.40 s), indicating occasional outlier responses with substantially elevated latency. The histogram for this scenario (*see Figure 9*) shows that ChatGPT and Claude remained relatively coherent in both mean and spread. In contrast, Gemini and, especially, Grok showed heavier tails and less predictable timing under dense multi-agent conditions.

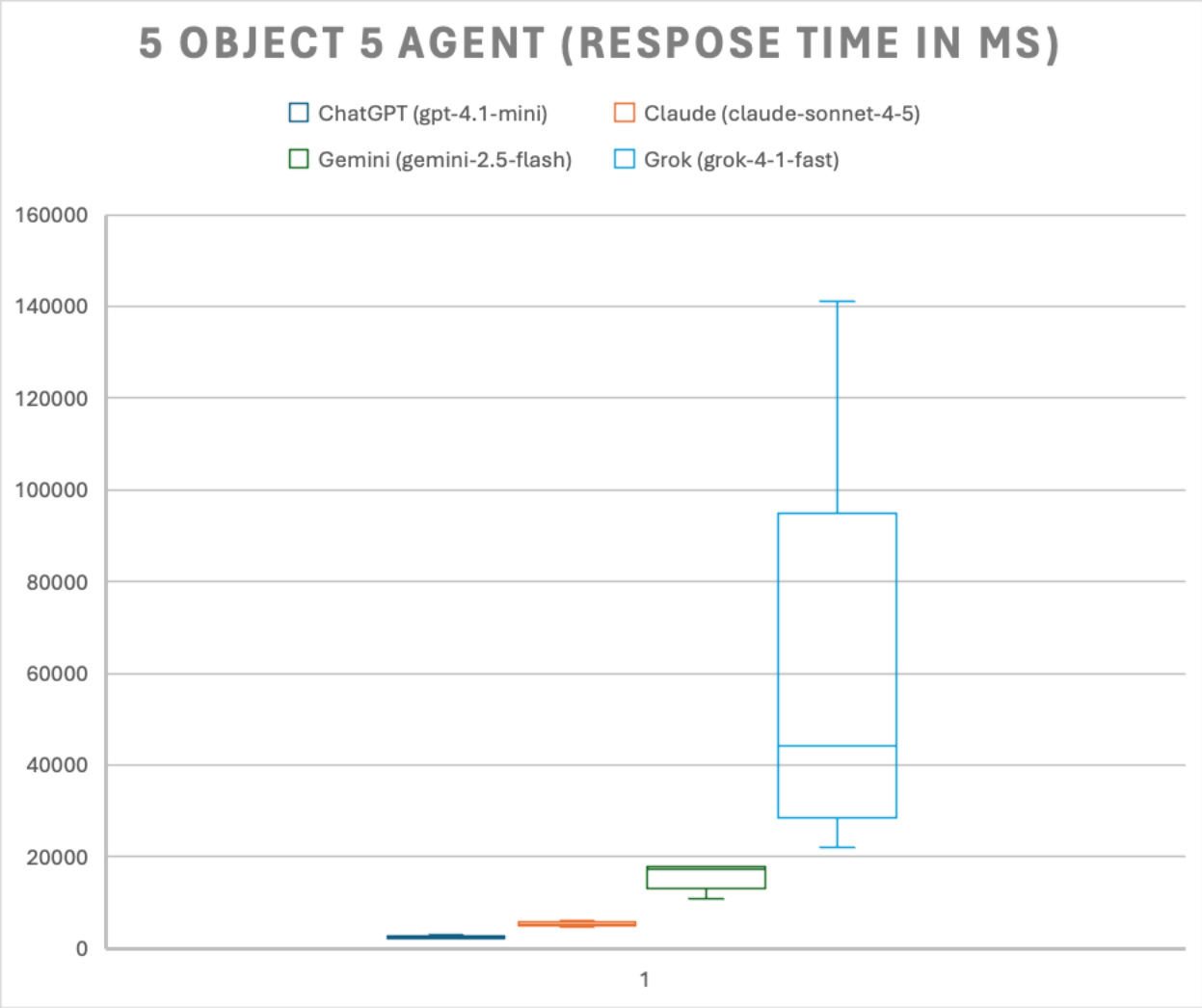


Figure 9: Boxplots showing response time for scenario 4.

4.5 Scenario 5: 10 Objects, 5 Agents

The final scenario represented the highest tested complexity, with five agents interacting within a scene containing ten objects. As with all previous scenarios, the outputs from all four models were structurally valid, followed the prompt's constraints, and were fully parsable and executable by the SceneDirector. The resulting action sequences maintained narrative coherence and respected the interaction rules encoded in the system prompt. Representative outputs for each model are provided in Appendix A1.5.

Table 7: Processing and response time scenario 5. Data are measured in milliseconds.

	ChatGPT	**Claude**	**Gemini**	**Grok**
	2236.45	6596.39	17348.49	61901.68
	1883.59	6997.91	9566.51	33531.24
	2753.76	4309.32	15278.96	36185.26
	2085.47	4833.90	16666.56	32192.96
	2602.16	6433.9457	10621.5779	39197.48
Average (s)	2.31	5.83	13.90	40.60
SD (s)	0.36	1.19	3.57	12.21

As shown in *Table 7,* in this scenario, ChatGPT achieved a mean response time of 2.31 s (SD = 0.36 s), indicating that its latency remained relatively low and stable even at the highest complexity level. Claude's mean response time increased further to 5.83 s (SD = 1.19 s). Gemini's average latency was 13.90 s (SD = 3.57 s), lower than in the 5-object/5-agent case but still substantially higher than ChatGPT and Claude. Grok's mean response time decreased relative to Scenario 4 but remained high at 40.60 s (SD = 12.21 s). The histogram for this scenario (*see Figure 10*) shows that ChatGPT and Claude again exhibited comparatively tight distributions, whereas Gemini and Grok continued to display broader spreads, reflecting greater variability as scene complexity increased.

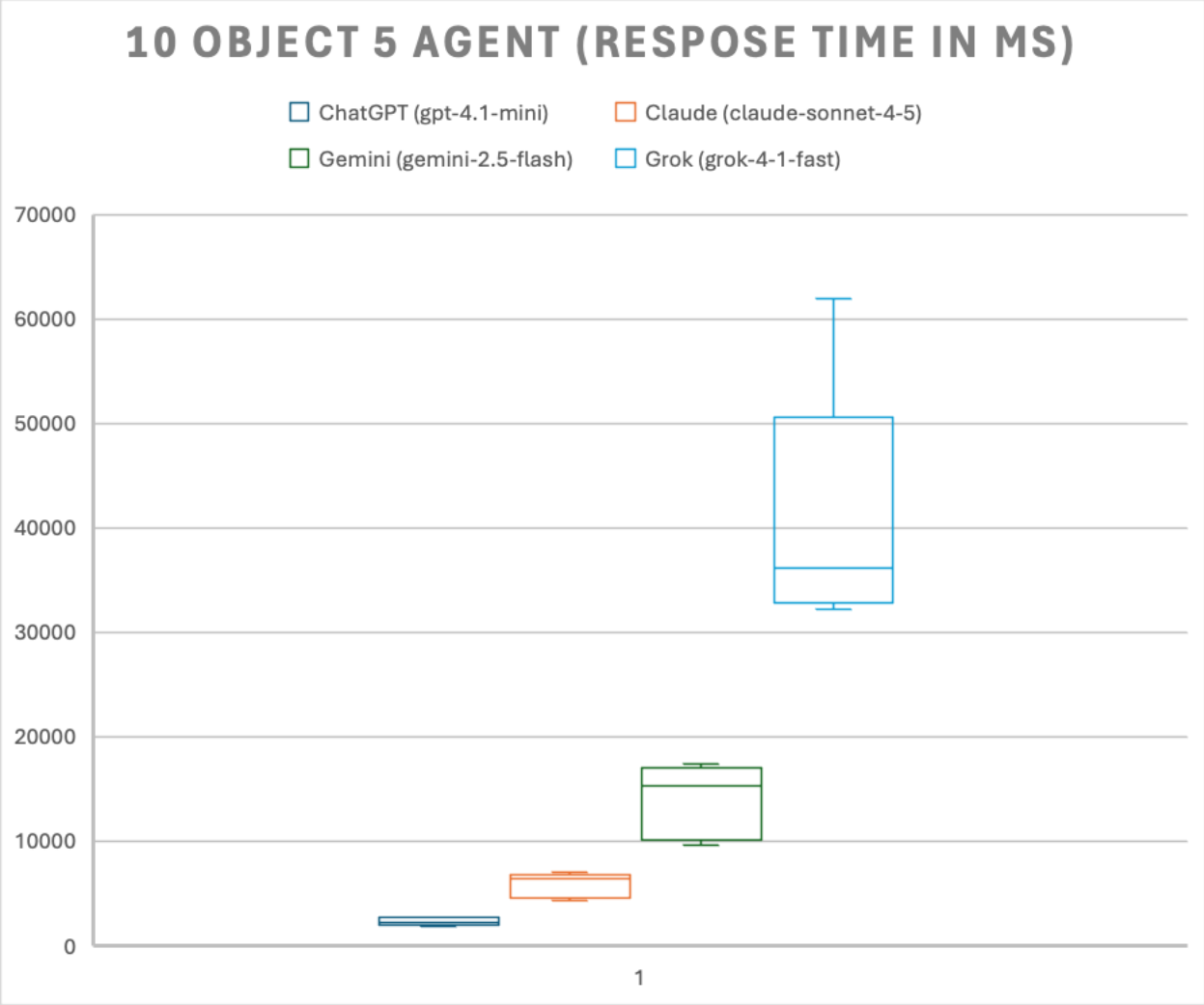


Figure 10: Boxplots showing response time for scenario 5.

4.6 Overall Comparison Across Scenarios

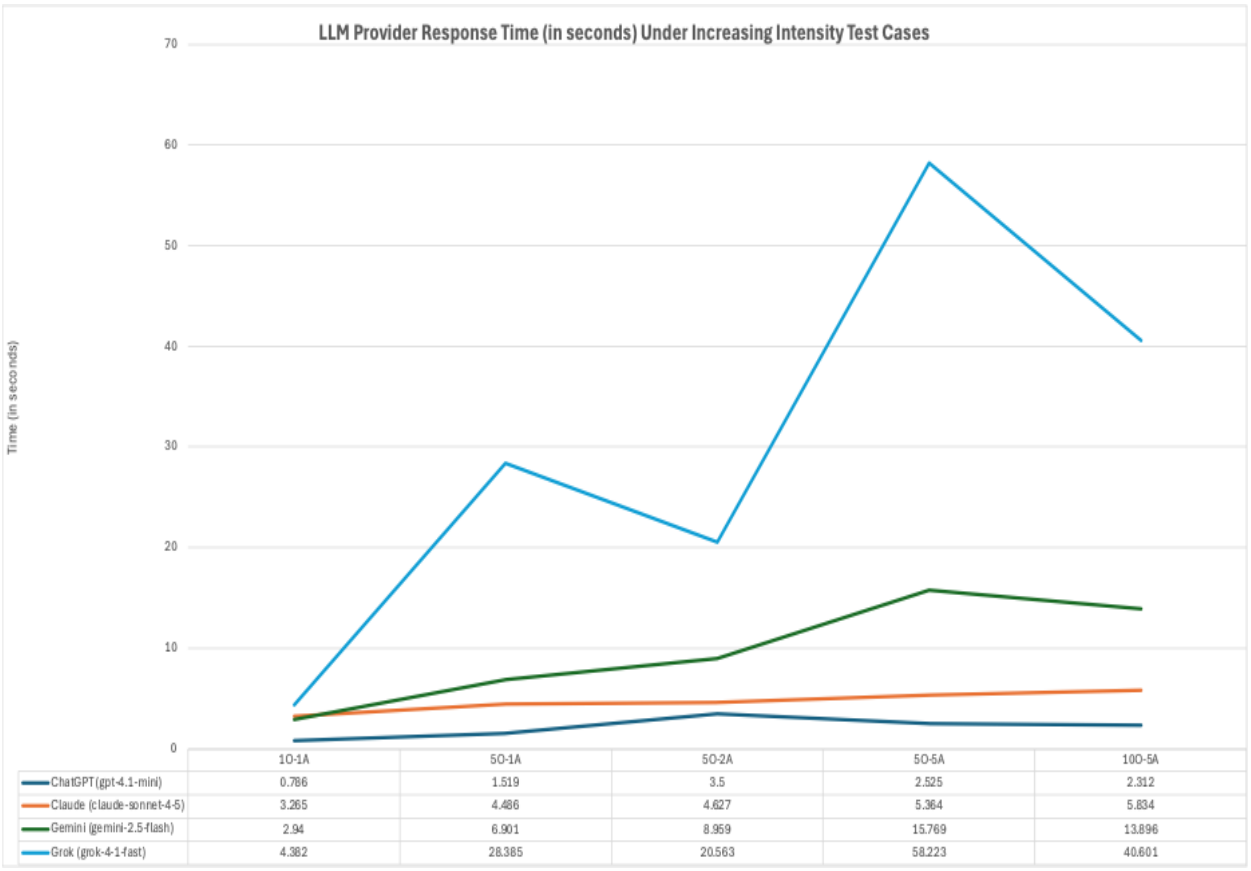
To compare model behavior across all complexity levels, the average response times per scenario were aggregated using the summary values shown in the final comparison sheet (see Table 8 *and Figure 11 for the line graph)*. Across all five scenarios, ChatGPT (gpt-4.1-mini) consistently exhibited the lowest latency, with mean response times ranging from 0.79 s in the simplest configuration (1O–1A) to around 3.50 s in the slightly more demanding multi-agent setup (5O–2A), and remaining close to 2–2.5 s in the higher-object-count scenarios (5O–5A and 10O–5A). Claude (claude-sonnet-4-5) formed a second, slightly slower tier, with mean response times ranging from 3.27 s to 5.83 s across scenarios and a generally gradual increase as complexity rose.

Table 8: Overall response time over scenarios.

	ChatGPT (gpt-4.1-mini)	Claude (claude-sonnet-4-5)	Gemini (gemini-2.5-flash)	Grok (grok-4-1-fast)
1O-1A	0.79	3.27	2.94	4.38
5O-1A	1.52	4.49	6.99	28.38
5O-2A	3.5	4.63	8.96	20.56
5O-5A	2.53	5.36	15.77	58.22
10O-5A	2.31	5.83	13.90	40.60

Gemini (gemini-2.5-flash) consistently lagged behind ChatGPT and Claude, with mean response times spanning 2.94 s to 15.77 s, showing a clear upward trend with increasing scene complexity, particularly in multi-agent settings. Grok (grok-4-1-fast) was the slowest model across all scenarios, with average response times ranging from 4.38 s in the simplest case to over 58 s in the 5-object/5-agent configuration, and remaining above 20 s even in the intermediate-complexity conditions. The overall line plot (*Figure 11*) illustrates a clear separation between ChatGPT and Claude on one hand, and Gemini and Grok on the other, in terms of both responsiveness and scalability with respect to scene complexity.

Across all scenarios, qualitative inspection confirmed that every retained output was structurally valid, adhered to the system prompt's constraints, and produced a coherent action sequence that could be executed in Unity. Representative SceneDirector strings for each model and scenario are provided in the appendix, illustrating that the primary differences between models in this study were in response time and variability rather than correctness or parsability.



	10-1A	50-1A	50-2A	50-5A	100-5A
ChatGPT (gpt-4.1-mini)	0.786	1.519	3.5	2.525	2.312
Claude (claude-sonnet-4-5)	3.265	4.486	4.627	5.364	5.834
Gemini (gemini-2.5-flash)	2.94	6.901	8.959	15.769	13.896
Grok (grok-4-1-fast)	4.382	28.385	20.563	58.223	40.601

Figure 11: Line graph depicting the response time of LLMs over increasing multi-agent scenario complexity.

5. CONCLUSION

This thesis investigated how natural language-driven narrative planning can be integrated with real-time behavior orchestration inside a modular 3D environment, addressing the central research question of whether large language models can translate high-level scene descriptions into executable multi-agent behaviors without requiring manual scripting. The system developed in this project demonstrates that such integration is feasible: by combining structured prefab metadata, scene-aware prompt construction, and a custom SceneDirector parser, the system successfully generates coherent, multi-agent interaction sequences that are executed autonomously in Unity.

The system allowed users to construct scenes via a drag-and-drop interface and automatically transformed those configurations into procedural story sequences that included navigation, animation, and object manipulation. Across all tested scenarios, the selected LLMs: ChatGPT, Claude, Gemini, and Grok consistently produced valid, parsable logic strings that adhered to the constraints imposed by the prompt design. The comparative analysis showed differences in response time and scaling behavior with increasing complexity, yet all models remained capable of producing structurally coherent action plans. Taken together, these results validate the viability of lightweight transformer-based models for procedural storytelling and confirm the effectiveness of the overall framework for rapid narrative prototyping.

5.1 Discussion: RQ1

The outcomes of this work illustrate both the strengths and the boundaries of LLM-driven procedural narrative generation. One significant finding is that scene-aware prompt design, combined with explicit formatting constraints, allows even lightweight transformer models to produce highly structured outputs suitable for runtime execution. This suggests that large-scale memory, formal symbolic planners, or autonomous reasoning layers are not strictly necessary for many forms of interactive story realization when the goal is short, self-contained sequences grounded in spatial context.

At the same time, the system exposes limitations inherent to working with LLMs as real-time logic generators. All decision-making remains externalized to the model, without persistent

state tracking. Although all model outputs were structurally valid in this study, their reliability depends heavily on prompt conditioning and careful metadata design. The system also inherits the latency constraints of remote API calls, making large-scale or time-sensitive applications challenging without optimizations. Despite these constraints, the evaluation demonstrates that LLMs provide a promising middle ground between manually authored behavior systems and fully autonomous agent architectures, especially in contexts that require fast iteration, flexible authoring, and multi-agent coordination without complex scripting.

5.2 Future Work

Several avenues exist to extend and strengthen the framework developed in this thesis. A natural next step is to introduce optional layers of agent autonomy, such as short-term memory, goal models, or reactive decision-making that can augment or refine the LLM-generated plan during runtime. Integrating dialogue generation or emotional modeling would also expand the system's expressive capacity, enabling richer narrative structures that go beyond movement and object interaction.

From a technical perspective, future work may focus on expanding the system’s interaction and animation capabilities to support richer and more varied behaviors. Enhancing the animation library and introducing physics-aware interactions would enable agents to engage with the environment more nuanced and contextually. Additional features, such as inline dialogue sequences or coordinated multimodal actions, could extend the expressive range of the generated narratives.

Model-side improvements also present opportunities for advancement. Incorporating local or fine-tuned language models can reduce latency, improve determinism, and make the framework better suited for real-time or offline applications. As generative AI continues to evolve, integrating more sophisticated planning mechanisms and expanding the system’s behavior repertoire would enable support for increasingly complex scenes and multi-agent interactions, thereby strengthening its applicability across diverse storytelling and simulation domains.

REFERENCES

[1] C. Liu, H. Wu, Y. Zhong, X. Zhang, Y. Wang, and W. Xie, "Intelligent grimm - Open-ended visual storytelling via latent diffusion models," in *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, IEEE, Jun. 2024, pp. 6190–6200. Accessed: Apr. 13, 2025. [Online]. Available: https://doi.org/10.1109/cvpr52733.2024.00592

[2] B. Kybartas and R. Bidarra, "A survey on story generation techniques for authoring computational narratives," *IEEE Transactions on Computational Intelligence and AI in Games*, vol. 9, no. 3, pp. 239–253, Sep. 2017, doi: 10.1109/tciaig.2016.2546063.

[3] M. Kapadia, S. Frey, A. Shoulson, R. W. Sumner, and M. Gross, "CANVAS: Computer-assisted narrative animation synthesis," *Eurographics/ACM SIGGRAPH Symposium on Computer Animation*, vol. 2016, no. 2016, pp. 199–209, 2016.

[4] J. H. Liew, H. Yan, J. Zhang, Z. Xu, and J. Feng, "MagicEdit: High-Fidelity and temporally coherent video editing," arXiv.org. [Online]. Available: https://arxiv.org/abs/2308.14749

[5] P. Ammanabrolu *et al.*, "Story realization: Expanding plot events into sentences," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, no. 05, pp. 7375–7382, Apr. 2020, doi: 10.1609/aaai.v34i05.6232.

[6] J. S. Park, J. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Generative agents: Interactive simulacra of human behavior," in *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, New York, NY, USA: ACM, Oct. 2023, pp. 1–22. Accessed: Apr. 13, 2025. [Online]. Available: https://doi.org/10.1145/3586183.3606763

[7] C. Qian *et al.*, "ChatDev: Communicative agents for software development," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Stroudsburg, PA, USA: Association for Computational Linguistics, 2024, pp. 15174–15186. Accessed: Apr. 13, 2025. [Online]. Available: https://doi.org/10.18653/v1/2024.acl-long.810

[8] B. Ning and M. Pei, "Task and environment-aware virtual scene rearrangement for enhanced safety in virtual reality," *IEEE Transactions on Visualization and Computer Graphics*, vol. 30, no. 5, pp. 2517–2526, May 2024, doi: 10.1109/tvcg.2024.3372115.

[9] A. Normoyle, J. Sedoc, and F. Durupinar, "Using llms to animate interactive story characters with emotions and personality," in *2024 IEEE Conference on Virtual Reality and 3D User Interfaces Abstracts and Workshops (VRW)*, IEEE, Mar. 2024, pp. 632–635. Accessed: Apr. 14, 2025. [Online]. Available: https://doi.org/10.1109/vrw62533.2024.00124

[10] D. Li, S. S. Sohn, S. Zhang, C.-J. Chang, and M. Kapadia, "From words to worlds: Transforming one-line prompts into multi-modal digital stories with LLM agents," in *The 17th ACM SIGGRAPH Conference on Motion, Interaction, and Games*, New York, NY, USA: ACM, Nov. 2024, pp. 1–12. Accessed: Apr. 14, 2025. [Online]. Available: https://doi.org/10.1145/3677388.3696321

[11] W. Li, C. Li, M. Kim, H. Huang, and L.-F. Yu, "Location-Aware adaptation of augmented reality narratives," in *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*, New York, NY, USA: ACM, Apr. 2023, pp. 1–15. Accessed: Apr. 14, 2025. [Online]. Available: https://doi.org/10.1145/3544548.3580978

[12] C. Li and L.-F. Yu, "Generating activity snippets by learning human-scene interactions," *ACM Transactions on Graphics*, vol. 42, no. 4, pp. 1–15, Jul. 2023, doi: 10.1145/3592096.

[13] C. Li, W. Li, H. Huang, and L.-F. Yu, "Interactive augmented reality storytelling guided by scene semantics," *ACM Transactions on Graphics*, vol. 41, no. 4, pp. 1–15, Jul. 2022, doi: 10.1145/3528223.3530061.

[14] K. He, A. Lapham, and Z. Li, "Enhancing Narratives with SayMotion's text-to-3D animation and LLMs," in *Special Interest Group on Computer Graphics and Interactive Techniques Conference Real-Time Live!*, New York, NY, USA: ACM, Jul. 2024, pp. 1–2. Accessed: Apr. 14, 2025. [Online]. Available: https://doi.org/10.1145/3641520.3665309

[15] A. Rychert, M. L. Ganuza, and M. N. Selzer, "Integrating GPT as an assistant for low-cost virtual reality escape-room games," *IEEE Computer Graphics and Applications*, vol. 44, no. 4, pp. 14–25, Jul. 2024, doi: 10.1109/mcg.2024.3426314.

[16] C. Hu, R. Wu, and X. Deng, "Coordination of NPCs in multi-agent systems based on behavior trees," *Applied and Computational Engineering*, vol. 95, no. 1, pp. 126–140, Oct. 2024, doi: 10.54254/2755-2721/95/2024melb0068.

[17] M. Kapadia, S. Singh, G. Reinman, and P. Faloutsos, "A Behavior-Authoring Framework for Multiactor Simulations," *IEEE Computer Graphics and Applications*, vol. 31, no. 6, pp. 45–55, Nov. 2011, doi: 10.1109/mcg.2011.68.

[18] C.-J. Chang, Q. T. Liu, H. Zhou, V. Pavlovic, and M. Kapadia, "CASIM: Composite Aware Semantic Injection for Text to Motion Generation," arXiv.org. [Online]. Available: https://arxiv.org/abs/2502.02063

[19] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Boston, MA, USA: Pearson, 2021.

[20] J. Eisenstein, *Introduction to Natural Language Processing*. Cambridge, MA, USA: MIT Press, 2019.

[21] N. Shaker, J. Togelius, and M. J. Nelson, *Procedural Content Generation in Games: A Textbook and an Overview of Current Research*. Cham, Switzerland: Springer, 2016.

[22] M. Wooldridge, *An Introduction to MultiAgent Systems*, 2nd ed. Hoboken, NJ, USA: Wiley, 2009.

[23] B. Li, S. Lee-Urban, and M. O. Riedl, "Learning goal-driven autonomy from human players in open-world RPGs," in *Proc. 7th AAAI Conf. Artif. Intell. Interactive Digit. Entertainment (AIIDE)*, Palo Alto, CA, USA, Oct. 2011, pp. 14–19.

[24] Liu, H., Choi, M., Kao, D., & Mousas, C. (2023). Synthesizing game levels for collaborative gameplay in a shared virtual environment. ACM Transactions on Interactive Intelligent Systems, 13(1), 1-36.

[25] Mazumdar, A., & Mousas, C. (2021). Synthesizing affective virtual reality multicharacter experiences. Computer Animation and Virtual Worlds, 32(3-4), e2004.

[26] Nelson, M., & Mousas, C. (2020, December). A Virtual Reality Framework for Human-Virtual Crowd Interaction Studies. In 2020 IEEE International Conference on Artificial Intelligence and Virtual Reality (AIVR) (pp. 209-213). IEEE.

[27] Koilias, A., Nelson, M. G., Anagnostopoulos, C. N., & Mousas, C. (2020). Immersive walking in a virtual crowd: The effects of the density, speed, and direction of a virtual crowd on human movement behavior. Computer Animation and Virtual Worlds, 31(6), e1928.

[28] Trivedi, H., & Mousas, C. (2023). Human-virtual crowd interaction: Towards understanding the effects of crowd avoidance proximity in an immersive virtual environment. Computer Animation and Virtual Worlds, 34(3-4), e2169.

[29] J. M. Fraga, A. Tsioliaridou, K. Chaudhuri, J. Casas, and S. Behnke, "PiGraph: Learning Interaction Dynamics with Motion Graphs and Neural Fields," *arXiv.org*. Accessed: Apr. 14, 2025. [Online]. Available: https://arxiv.org/abs/2405.07314

[30] C. Yu, S. Starke, A. Liu, H. Lv, and H. Li, "Understanding and Exploiting Object Interaction Landscapes," *ACM Transactions on Graphics*, vol. 43, no. 4, pp. 1–16, Aug. 2024. doi: 10.1145/3658203.

[31] A. Bhatt, B. Wang, S. Moghadam, and J. C. Niebles, "Kimera: Generative Motion Planning for Human–Object Interaction," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, IEEE, 2024, pp. 4078–4087. doi: 10.1109/CVPR52661.2024.00402.

APPENDIX

1. Appendix A: LLM responses

Scenario 1: 1 Object 1 Agent

Example: https://youtu.be/0Ho2UC-AS2o

Scene Description

```
1. Scene Description:
2. ----------
3. Actors:
4. ----------
5. Name: Guy
6. ID: H_1
7. Tags: male, college student, casual, claustrophobic
8. Position: (11.00, 0.01, 5.53)
9. ----------
10. ----------
11. Interactable Objects:
12. ----------
13. Object ID: Obj_0
14. Name: Light Switch
15. Is Grabbable: No
16. Is Stationary: No
17. Is Stationary Compatible: No
18. Is Basic Interaction :Yes
19. Tags: room-light, light-switch
20. Position: (-0.68, 2.95, -1.83)
21. ----------
22. ----------
23.      END
24. ----------
```

ChatGPT

H_1{Obj_0(T, 3, 1.5, F, F, T)}

Claude

H_1{Obj_0(T, 2, 1.2, F, F, T)}

Gemini

H_1{Obj_0(T, 3, 1.0, F, F, T)}

Grok

H_1{Obj_0(T, 3, 1.5, F, F, T)}

Scenario 2: 5 Object 1 Agent

Example: https://youtu.be/CVlQcsojeLU

Scene Description:

```
1. Scene Description:
2. ----------
3. Actors:
4. ----------
5. Name: Guy
6. ID: H_1
7. Tags: male, college student, casual, claustrophobic
8. Position: (10.00, 0.01, 2.53)
9. ----------
10. ----------
11. Interactable Objects:
12. ----------
13. Object ID: Obj_4
14. Name: Washing Machine
15. Is Grabbable: No
16. Is Stationary: No
17. Is Stationary Compatible: No
18. Is Basic Interaction :Yes
19. Tags: Wash, Clean
20. Position: (15.67, 0.00, 1.95)
21. ----------
22. Object ID: Obj_3
23. Name: Phone
24. Is Grabbable: Yes
25. Is Stationary: No
26. Is Stationary Compatible: No
27. Is Basic Interaction :No
28. Tags: call, phone, mobile
29. Position: (16.70, 1.23, 8.44)
30. ----------
31. Object ID: Obj_2
32. Name: Computer
```

```
33. Is Grabbable: No
34. Is Stationary: Yes
35. Is Stationary Compatible: No
36. Is Basic Interaction :No
37. Tags: computer, work, play
38. Position: (15.62, 0.89, 8.71)
39. ----------
40. Object ID: Obj_1
41. Name: Bed
42. Is Grabbable: No
43. Is Stationary: Yes
44. Is Stationary Compatible: No
45. Is Basic Interaction :No
46. Tags: bed, sleep, rest
47. Position: (7.14, 0.60, 11.53)
48. ----------
49. Object ID: Obj_0
50. Name: Light Switch
51. Is Grabbable: No
52. Is Stationary: No
53. Is Stationary Compatible: No
54. Is Basic Interaction :Yes
55. Tags: room-light, light-switch
56. Position: (-0.68, 2.95, -1.83)
57. ----------
58. ----------
59.      END
60. ----------
```

ChatGPT

H_1{Obj_0(T, 3, 1, F, F, T), Obj_3(T, 5, 1.5, T, F, F), Obj_2(T, 10, 1.2, F, T, F), Obj_4(T, 4, 1, F, F, T), Obj_0(T, 3, 1, F, F, T), Obj_1(T, 6, 1, F, T, F)}

Claude

H_1{Obj_0(T, 2, 1.2, F, F, T), Obj_3(T, 4, 1.5, T, F, F), Obj_2(T, 8, 1.3, F, T, F), Obj_4(T, 3, 1.2, F, F, T), Obj_1(T, 12, 1.0, F, T, F), Obj_0(T, 2, 1.2, F, F, T)}

Gemini

H_1{Obj_1(T, 5, 1.5, F, T, F), Obj_0(T, 3, 1.0, F, F, T), Obj_3(T, 8, 1.2, T, F, F), Obj_2(T, 12, 1.0, F, T, F), Obj_4(T, 4, 1.0, F, F, T), Obj_1(T, 6, 1.0, F, T, F)}

Grok

H_1{Obj_0(T,3,2.5,F,F,T), Obj_4(T,4,2.0,F,F,T), Obj_2(T,10,1.5,F,T,F), Obj_3(T,4,1.5,T,F,F), Obj_1(T,12,1.2,F,T,F)}

Scenario 3: 5 Object 2 Agent

Example: https://youtu.be/0P8ugBlgAV0

Scene Description

```
1. Scene Description:
2. ----------
3. Actors:
4. ----------
5. Name: Girl
6. ID: H_2
7. Tags: Female, college student, casual, claustrophobic
8. Position: (14.00, 0.01, -0.67)
9. ----------
10. Name: Guy
11. ID: H_1
12. Tags: male, college student, casual, claustrophobic
13. Position: (6.00, 0.01, 6.33)
14. ----------
15. ----------
16. Interactable Objects:
17. ----------
18. Object ID: Obj_4
19. Name: Television
20. Is Grabbable: No
21. Is Stationary: No
22. Is Stationary Compatible: No
23. Is Basic Interaction :Yes
24. Tags: Television, TV, entertainment
25. Position: (11.63, -0.27, 5.25)
26. ----------
27. Object ID: Obj_3
28. Name: Sofa
29. Is Grabbable: No
30. Is Stationary: Yes
31. Is Stationary Compatible: No
32. Is Basic Interaction :No
33. Tags: sofa, sit, rest, couch
34. Position: (13.65, 0.61, 4.43)
```

```
35. ----------
36. Object ID: Obj_2
37. Name: Lamp
38. Is Grabbable: No
39. Is Stationary: No
40. Is Stationary Compatible: No
41. Is Basic Interaction :Yes
42. Tags: light, night-light, lamp
43. Position: (12.00, 1.29, 11.00)
44. ----------
45. Object ID: Obj_1
46. Name: Bed
47. Is Grabbable: No
48. Is Stationary: Yes
49. Is Stationary Compatible: No
50. Is Basic Interaction :No
51. Tags: bed, sleep, rest
52. Position: (8.14, 0.60, 10.53)
53. ----------
54. Object ID: Obj_0
55. Name: Light Switch
56. Is Grabbable: No
57. Is Stationary: No
58. Is Stationary Compatible: No
59. Is Basic Interaction :Yes
60. Tags: room-light, light-switch
61. Position: (-0.68, 2.95, -1.83)
62. ----------
63. ----------
64.      END
65. ----------
```

ChatGPT

H_1{Obj_0(T, 3, 2, F, F, T), Obj_4(T, 5, 3, F, F, T), Obj_3(T, 5, 2, F, T, F)}, H_2{Obj_3(T, 5, 3, F, T, F), Obj_2(T, 3, 2, F, F, T), Obj_1(T, 6, 2, F, T, F), Obj_0(T, 3, 3, F, F, T)}

Claude

H_1{Obj_0(T, 2, 1.2, F, F, T), Obj_4(T, 3, 1.0, F, F, T), Obj_3(T, 8, 1.0, F, T, F), Obj_1(T, 12, 1.0, F, T, F), Obj_0(T, 2, 1.0, F, F, T)}, H_2{Obj_2(T, 2, 1.5, F, F, T), Obj_3(T, 10, 1.2, F, T, F), Obj_1(T, 14, 1.0, F, T, F)}

Gemini

H_1{Obj_0(T, 3.5, 1.5, F, F, T), Obj_1(T, 10, 1.5, F, T, F)}, H_2{Obj_3(T, 8, 1.5, F, T, F), Obj_4(T, 4, 1.5, F, F, T)}

Grok

H_1{Obj_0(T,5,3.5,F,F,T), Obj_4(T,5,2.5,F,F,T)}, H_2{Obj_3(T,12,1.5,F,T,F)}

Scenario 4: 5 Object 5 Agent

Example: https://youtu.be/DRuY1_UwCSE

Scene Description

```
1. Scene Description:
2. ----------
3. Actors:
4. ----------
5. Name: Old Male
6. ID: H_5
7. Tags: Male, retired, old, PTSD
8. Position: (4.00, 0.01, 4.53)
9. ----------
10. Name: Kid Male
11. ID: H_4
12. Tags: Male, elementary student, childish, nyctophobic
13. Position: (5.00, 0.01, 0.33)
14. ----------
15. Name: Kid Female
16. ID: H_3
17. Tags: Female, elementary student, childish, nyctophobic
18. Position: (16.00, 0.01, 9.33)
19. ----------
20. Name: Girl
21. ID: H_2
22. Tags: Female, college student, casual, claustrophobic
23. Position: (14.00, 0.01, -0.67)
24. ----------
25. Name: Guy
26. ID: H_1
27. Tags: male, college student, casual, claustrophobic
28. Position: (5.96, 0.01, 6.37)
29. ----------
30. ----------
31. Interactable Objects:
```

```
----------
Object ID: Obj_4
Name: Television
Is Grabbable: No
Is Stationary: No
Is Stationary Compatible: No
Is Basic Interaction :Yes
Tags: Television, TV, entertainment
Position: (11.63, -0.27, 5.25)
----------
Object ID: Obj_3
Name: Sofa
Is Grabbable: No
Is Stationary: Yes
Is Stationary Compatible: No
Is Basic Interaction :No
Tags: sofa, sit, rest, couch
Position: (13.65, 0.61, 4.43)
----------
Object ID: Obj_2
Name: Lamp
Is Grabbable: No
Is Stationary: No
Is Stationary Compatible: No
Is Basic Interaction :Yes
Tags: light, night-light, lamp
Position: (12.00, 1.29, 11.00)
----------
Object ID: Obj_1
Name: Bed
Is Grabbable: No
Is Stationary: Yes
Is Stationary Compatible: No
Is Basic Interaction :No
Tags: bed, sleep, rest
Position: (8.14, 0.60, 10.53)
----------
Object ID: Obj_0
Name: Light Switch
Is Grabbable: No
Is Stationary: No
Is Stationary Compatible: No
Is Basic Interaction :Yes
```

```
75. Tags: room-light, light-switch
76. Position: (-0.68, 2.95, -1.83)
77. ----------
78. ----------
79.      END
80. ----------
```

ChatGPT

H_1{Obj_0(T, 3.5, 2.5, F, F, T), Obj_4(T, 5, 1.5, F, F, T)}, H_2{Obj_3(T, 6, 1.3, F, T, F)}, H_3{Obj_2(T, 4, 2, F, F, T)}, H_4{Obj_1(T, 7, 1, F, T, F)}, H_5{Obj_0(T, 3.5, 2.5, F, F, T)}

Claude

H_1{Obj_0(T, 2, 1.5, F, F, T), Obj_2(T, 2, 1.2, F, F, T), Obj_3(T, 8, 1.0, F, T, F)}, H_2{Obj_4(T, 3, 1.5, F, F, T), Obj_3(T, 12, 1.0, F, T, F)}, H_5{Obj_1(T, 14, 1.0, F, T, F)}, H_3{Obj_2(T, 2, 1.8, F, F, T)}, H_4{Obj_0(F, 2, 2.0, F, F, T)}

Gemini

H_1{Obj_0(T, 3, 1.5, F, F, T), Obj_0(T, 3, 1.5, F, F, T)}, H_2{Obj_4(T, 4, 1.5, F, F, T), Obj_3(T, 8, 1.5, F, T, F)}, H_3{Obj_1(T, 12, 1.0, F, T, F)}, H_5{Obj_3(T, 10, 1.0, F, T, F)}, H_4{Obj_2(T, 4, 1.5, F, F, T)}

Grok

H_1{Obj_3(T,10,1.5,F,T,F)},H_2{Obj_1(T,12,1.5,F,T,F)},H_3{Obj_2(T,3.5,2.5,F,F,T)},H_4{Obj_4(T,4,2.5,F,F,T)},H_5{Obj_0(T,4,1.2,F,F,T)}

Scenario 5: 10 Object 5 Agent

Example: https://youtu.be/6ZsatigdKLE

Scene Description

```
 1. Scene Description:
 2. ----------
 3. Actors:
 4. ----------
 5. Name: Old Male
 6. ID: H_5
 7. Tags: Male, retired, old, PTSD
 8. Position: (6.00, 0.01, 0.33)
 9. ----------
10. Name: Kid Male
```

```
ID: H_4
Tags: Male, elementary student, childish, nyctophobic
Position: (1.80, 0.01, 11.53)
----------
Name: Kid Female
ID: H_3
Tags: Female, elementary student, childish, nyctophobic
Position: (9.82, 0.01, 7.35)
----------
Name: Girl
ID: H_2
Tags: Female, college student, casual, claustrophobic
Position: (13.53, 0.01, 7.86)
----------
Name: Guy
ID: H_1
Tags: male, college student, casual, claustrophobic
Position: (17.84, 0.01, 8.48)
----------
----------
Interactable Objects:
----------
Object ID: Obj_9
Name: Washing Machine
Is Grabbable: No
Is Stationary: No
Is Stationary Compatible: No
Is Basic Interaction :Yes
Tags: Wash, Clean
Position: (3.32, 0.00, 4.95)
----------
Object ID: Obj_8
Name: Phone
Is Grabbable: Yes
Is Stationary: No
Is Stationary Compatible: No
Is Basic Interaction :No
Tags: call, phone, mobile
Position: (16.70, 1.23, -2.56)
----------
Object ID: Obj_7
Name: Computer
Is Grabbable: No
```

```
Is Stationary: Yes
Is Stationary Compatible: No
Is Basic Interaction :No
Tags: computer, work, play
Position: (15.62, 0.89, -2.29)
----------
Object ID: Obj_6
Name: Chair
Is Grabbable: No
Is Stationary: Yes
Is Stationary Compatible: No
Is Basic Interaction :No
Tags: sit, chair, rest
Position: (11.00, 0.57, 0.99)
----------
Object ID: Obj_5
Name: Television
Is Grabbable: No
Is Stationary: No
Is Stationary Compatible: No
Is Basic Interaction :Yes
Tags: Television, TV, entertainment
Position: (9.63, -0.27, 4.25)
----------
Object ID: Obj_4
Name: Sofa
Is Grabbable: No
Is Stationary: Yes
Is Stationary Compatible: No
Is Basic Interaction :No
Tags: sofa, sit, rest, couch
Position: (13.65, 0.61, 3.43)
----------
Object ID: Obj_3
Name: Bed
Is Grabbable: No
Is Stationary: Yes
Is Stationary Compatible: No
Is Basic Interaction :No
Tags: bed, sleep, rest
Position: (14.14, 0.60, 11.53)
----------
Object ID: Obj_2
```

```
 97. Name: Bed
 98. Is Grabbable: No
 99. Is Stationary: Yes
100. Is Stationary Compatible: No
101. Is Basic Interaction :No
102. Tags: bed, sleep, rest
103. Position: (9.14, 0.60, 11.53)
104. ----------
105. Object ID: Obj_1
106. Name: Bed
107. Is Grabbable: No
108. Is Stationary: Yes
109. Is Stationary Compatible: No
110. Is Basic Interaction :No
111. Tags: bed, sleep, rest
112. Position: (4.14, 0.60, 11.53)
113. ----------
114. Object ID: Obj_0
115. Name: Light Switch
116. Is Grabbable: No
117. Is Stationary: No
118. Is Stationary Compatible: No
119. Is Basic Interaction :Yes
120. Tags: room-light, light-switch
121. Position: (-0.68, 2.95, -1.83)
122. ----------
123. ----------
124.      END
125. ----------
```

ChatGPT

H_4{Obj_0(T, 3, 3, F, F, T), Obj_1(T, 12, 1.5, F, T, F)}, H_3{Obj_5(T, 4, 2, F, F, T), Obj_2(T, 10, 1.5, F, T, F)}, H_2{Obj_6(T, 8, 1.8, F, T, F)}, H_1{Obj_8(T, 4, 3, T, F, F), Obj_7(T, 11, 1.5, F, T, F)}, H_5{Obj_9(T, 4, 2, F, F, T), Obj_4(T, 10, 1.2, F, T, F)}

Claude

H_5{Obj_0(T, 2, 1.5, F, F, T), Obj_9(T, 3, 1.2, F, F, T), Obj_6(T, 8, 1.0, F, T, F), Obj_5(T, 2, 1.0, F, F, T), Obj_4(T, 12, 1.0, F, T, F), Obj_0(T, 2, 1.0, F, F, T)}, H_4{Obj_1(T, 10, 1.2, F, T, F)}, H_3{Obj_2(T, 10, 1.2, F, T, F)}, H_2{Obj_3(T, 5, 1.5, F, T, F), Obj_7(T, 8, 1.3, F, T, F), Obj_8(T, 4, 1.2, T, F, F), Obj_3(T, 6, 1.0, F, T, F)}, H_1{Obj_3(F, 0, 1.0, F, F, F)}

Gemini

H_4{Obj_0(T, 4, 1.5, F, F, T), Obj_1(T, 12, 1.5, F, T, F)}, H_3{Obj_5(T, 4, 1.5, F, F, T), Obj_2(T, 12, 1.5, F, T, F)}, H_5{Obj_9(T, 5, 1.0, F, F, T), Obj_6(T, 8, 1.0, F, T, F)}, H_1{Obj_7(T, 10, 2.0, F, T, F), Obj_8(T, 6, 2.0, T, F, F)}, H_2{Obj_4(T, 8, 2.0, F, T, F)}

Grok

H_1{Obj_7(T,10,2.0,F,T,F)}, H_2{Obj_5(T,3,2.5,F,F,T), Obj_4(T,12,2.0,F,T,F)}, H_3{Obj_2(T,12,3.5,F,T,F)}, H_4{Obj_1(T,12,4.0,F,T,F)}, H_5{Obj_0(T,3,1.0,F,F,T), Obj_6(T,10,1.0,F,T,F)}